Current Prediction in Analog Integrated Circuits Using Graph Attention Networks

Matthew Jones
Oregon State University
Corvallis, Oregon
jonesm25@oregonstate.edu

Abstract

Modern place and route agents for analog integrated circuits are required to meet the design rules. For many interconnects this may be overkill, leading to wasted area. Determining which interconnects can be drawn with strategic design rule violations, without sacrificing reliability, requires knowledge of the current flowing through the interconnect. Running simulations on the full analog design is expensive, particularly late in the design cycle. This paper proposes a simpler way: a Deep Learning model that can predict the current based on the design netlist and manufacturing process models using Graph Attention Networks (GATs). This paper shows that a GATv2 model can predict inter-stage current with high accuracy, achieving a mean relative error (MRE) of $1.91\times$ *on a single-process dataset (22nm HP CMOS). This shows that deep learning can, in principle, replace SPICE simulations for current prediction in analog design. However, when generalizing to a multi-process dataset spanning nine process technologies, the error increases to* $7.7\times$*, indicating that more robustness and process-specific adaptation are needed for broad applicability. These promising results suggest that with further improvements in model robustness and generalization, deep learning approaches can become a practical tool for analog current prediction.*

1. Introduction

The motivation behind this project is to further research into the use of Deep Learning to minimize analog integrated circuit design area without sacrificing reliability. A recent review of the literature searching All Metadata for "Artificial Intelligence" or "Reinforcement Learning" or "Machine Learning," and "Design Rule" or "Layout" or "Route," in the ISSCC, the JSSC, the CICC, or the Symposium on VLSI Circuits resulted in only 10 papers in the last 10 years. Park twice wrote about ML-based layout optimization to control timing skews [13] [14] and Zhou wrote about using RL for design space discovery and end-to-end synthesis [19]. However, most wrote about IC designs and methodologies to improve AI model performance, such as Dong [5], Kaul [9], Everson [7], Zhao [17], Feng [8], Sinangil [15], and Cho [4]. None of the ten papers found in these top journals were about Deep Learning, IC Area, or Reliability. The long-term goal of this project is to change that. The short-term goal of the project is to prove it can be done.

1.1. Physical Design

Even those who are unfamiliar with the integrated circuit design process are familiar with the concept of a circuit schematic, which is a symbolic representation of the circuit's function. Physical design is the process of "drawing" the circuit elements how they are actually put on silicon. Individual circuits may be designed and used as "blocks" in other circuits. Connecting blocks within a larger circuit is the process of "placing" the blocks and then "routing" the connection wires between them. See Figure 1 for an example of a basic two-block circuit schematic and the corresponding physical design.

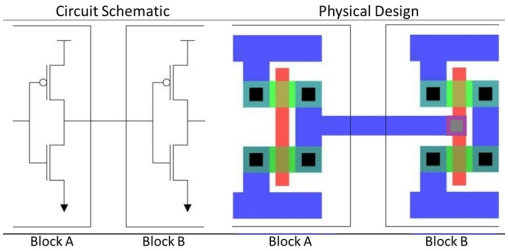

Figure 1: Circuit Example

Typically, these metal wires are drawn to meet the minimum width requirements in the design rules for the manufacturing process. However, if a design rule (DR) must be violated to avoid rework or save area, a waiver for the rule may be granted. Design rules can be overkill for metal lines with little current flowing through them. However, typical Reinforcement Learning-based place and route (P&R) agents are constrained by the design rules. Strategic design rule violations could potentially provide significant area savings without a significant impact on reliability. The key is to know which connection wires can violate the rules without sacrificing reliability.

1.2. Strategic DR Violation

The solution is to provide the Reinforcement Learning (RL) agent with the current flowing through each interconnection wire so that it can decide, based on current design calculations, which wires can be made narrower than the design rule. Unfortunately, creating a simulation testbench for each circuit is expensive. A solution to this expense is to use a Deep Learning model to estimate the current based on the design and process parameters, provide that information to the RL agent which then performs P&R, then calculate the circuit reliability based on the routed layout. This flowchart is shown in Figure 2.

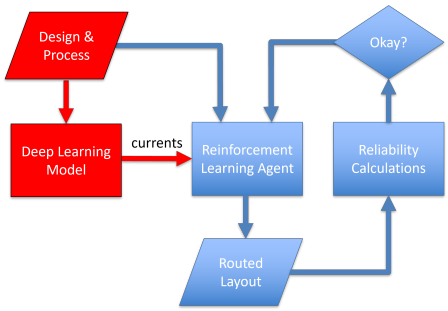

Figure 2: System Flowchart

The Deep Learning model needs to be trained on a dataset that contains simulation results representative of a circuit testbench.

1.3. Graph Neural Networks

Graph neural networks are a type of neural network that uses a complete graph structure, rather than a single data sample, as one input. Mathematically, the graph is represented as a weighted adjacency matrix and a feature matrix whose rows are the feature vectors of each individual node in the graph. Learning can then be done to do classification or regression on node labels, edge weights, or an aggregate graph level [18]. GNNs can consist of many different types of layers, but the one that is most central to the concept is known generally as the message passing layer. This layer updates the features at each node, using information pulled from neighboring node features. For a general GNN this is not strictly defined, although the edges typically encode how the 'messages' are passed between nodes.

While there are many types of GNNs, Graph Attention Networks (GATs) extend the basic message-passing framework by learning a scalar attention weight for each directed edge, so the model can adaptively weight the influence of each neighbour rather than treating all connections equally [16]. GATv2 [3] corrects a rank-limitation in the original GAT by making the attention function strictly more expressive. In GATv2, the attention score for a given edge (i, j) is computed as:

$$e_{ij} = \mathbf{a}^\top \mathrm{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \| \mathbf{h}_j]) \tag{1}$$

where:

- $\mathbf{h}_i$ and $\mathbf{h}_j$ are the feature vectors of the source and target nodes, respectively
- $[\mathbf{h}_i \| \mathbf{h}_j]$ denotes concatenation of the two feature vectors
- $\mathbf{W}$ is a learnable weight matrix applied to the concatenated features
- $\mathrm{LeakyReLU}$ is a non-linear activation function
- $\mathbf{a}$ is a learnable vector that projects the result to a scalar attention score

This formulation allows the attention mechanism to depend jointly on both nodes, making it more expressive than the original GAT, which used separate projections and summed them before the nonlinearity. In a circuit graph, nodes represent transistors and interconnects whose relative contributions to downstream current vary depending on operating point; learned per-edge attention weights are therefore a natural fit compared to the uniform aggregation of a plain GCN [10].

1.4. Related Work

There is already a large amount of existing literature on the topic of GNNs. For circuits specifically, El Sayed et al [6] conduct a survey on GNNs as tools in circuit design, although their scope is considerably broader than our own, discussing not just physical design but other applications such as

logic design and synthesis. For direct application, Mirhoseini et al use edge-based GCNs to solve optimization problems in physical design [12]. Their work once again differs from ours in the sense that the scope is much broader. Additionally, their approach differs in that they use deep reinforcement learning, while this is a simpler supervised learning approach.

The remainder of this report is organized as follows: Section 2 describes dataset generation, including the circuit description, simulation methodology, and the datasets that were created; Section 3 details the technical approach, including the Graph Neural Network and training procedure; Section 4 presents the experimental design and results; and Section 5 provides the main conclusions and future work. The source code is in the Appendix.

2. Datasets

The first step toward creating a dataset is to choose a test circuit. For this project, the test circuit is a Cascaded CMOS Inverter Chain, as shown in Figure 3. The Spice netlist () is in the Appendix. The target node, which represents the connection between two circuit blocks, is between them. Functionally, it is as simple as the circuit shown in Figure 1, but requires additional elements related to the testbench in order to capture the simulation results necessary to generate the dataset. The description of each element is below.

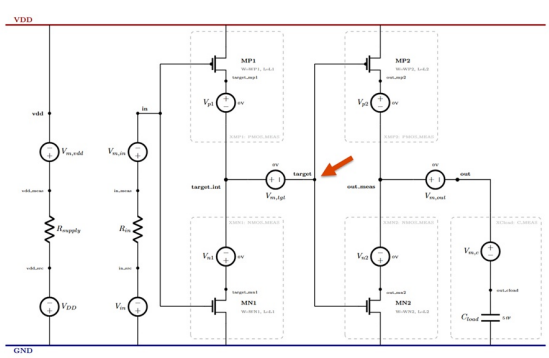

Figure 3: Cascaded Inverter Testbench (2inv.sp)

2.1. Circuit Description

The testbench is comprised of transistors, subcircuits, measurement sources, and supply and input sources. The transistors are what make up the inverters. The rest of the elements are to facilitate measurement for data capture. A description of each element is below.

Transistors

- *MP1* (PMOS, W=WP1, L=L1) is the pull-up transistor of the *driver inverter* (INV1). When the input is low, MP1 charges the `target` node toward VDD. Its width WP1 and length L1 are swept across the design space.
- *MN1* (NMOS, W=WN1, L=L1) is the pull-down transistor of the driver inverter. When the input is high, MN1 discharges `target` toward ground. Together with MP1, it sets the switching threshold and transition speed of INV1.
- *MP2* (PMOS, W=WP2, L=L2) is the pull-up transistor of the *load inverter* (INV2). Driven by the `target` node, it charges the `out` node. Its dimensions WP2 and L2 are independently swept.
- *MN2* (NMOS, W=WN2, L=L2) is the pull-down transistor of the load inverter. Together with MP2, it presents a realistic capacitive and resistive load to the driver inverter, which directly affects the peak current drawn from VDD.

Subcircuits

- *XMP1: PMOS_MEAS* (contains MP1 + V_{p1}) wraps the PMOS transistor with a 0 V measurement voltage source (V_{p1}) in series with its drain. This allows SPICE to report the instantaneous drain current of MP1 as a branch current without disturbing the circuit.
- *XMN1: NMOS_MEAS* (contains MN1 + V_{n1}) uses the same technique for the NMOS transistor of INV1, enabling measurement of MN1's drain current via V_{n1}.
- *XMP2: PMOS_MEAS* (contains MP2 + V_{p2}) is the rain current measurement wrapper for MP2 in the load inverter.
- *XMN2: NMOS_MEAS* (contains MN2 + V_{n2}) is the rain current measurement wrapper for MN2 in the load inverter.
- *XCload: C_MEAS* (contains C_{load} + $V_{m,c}$) wraps the 5 fF load capacitor with a 0 V measurement source. This captures the current flowing into the capacitive load, representing the dynamic charging/discharging current that a downstream stage would see.

Measurement Sources (0 V)

- $V_{m,vdd}$ (Vmeas_vdd) measures the total current drawn from the VDD supply rail. Its peak value (`I_vdd`) captures supply current behavior rele-

vant to power integrity and IR-drop risk.

- $V_{m,in}$ (Vmeas_in) measures the current delivered by the input source. Captures gate-tunneling and capacitive charging currents at the input node (`I_in`).
- $V_{m,tgt}$ (Vmeas_target) measures the current flowing from the driver inverter's output node (`target_int`) to the load inverter's gate (`target`). Records `I_target`, the primary prediction target for the DL model, reflecting the inter-stage drive current.
- $V_{m,out}$ (Vmeas_out) measures the current at the output of the load inverter (`I_out`), capturing the current flowing into the load capacitor path.
- V_{p1}, V_{n1}, V_{p2}, V_{n2} is internal to PMOS_MEAS / NMOS_MEAS subcircuits. Each measures the drain current of its respective transistor, enabling per-device current attribution.
- $V_{m,c}$ is internal to C_MEAS. Measures the capacitor charging current, distinguishing it from resistive leakage paths.

Supply and Input Sources

- V_{DD} is the DC voltage source providing the supply voltage. The value is swept across the design space to capture voltage-dependent behavior.
- R_{supply} is a small series resistance (0.01 Ω) in the VDD path, modeling interconnect resistance and aiding convergence.
- V_{in} is the pulsed voltage source (PULSE) generating the switching stimulus. Rise/fall times are 20 ps; pulse width is 1 ns with a 2 ns period.
- R_{in} is a small series resistance (0.01 Ω) in the input path, matching the VDD path topology.

The goal of the simulation is to measure the max voltage and peak current at each node in circuit, so it can be added to the dataset. The nodes are described below.

- *vdd* is a regulated supply rail, downstream of $V_{m,vdd}$. Voltage `V_vdd` is recorded to verify supply integrity; current through $V_{m,vdd}$ gives the supply current target.
- *in* is the gate input to INV1, downstream of $V_{m,in}$. Driven by a PULSE source (V_{in}) through R_{in}. Voltage `V_in` and current `I_in` are recorded.
- *target_int* is an internal output of INV1 (junction of MP1 and MN1 drain paths), before the measurement source $V_{m,tgt}$.
- *target* is the gate input to INV2, downstream of $V_{m,tgt}$. Voltage `V_target` and current `I_target` are recorded. This is the inter-stage node whose transition drives the peak supply current.
- *out_meas* is an internal output of INV2 (junction of MP2 and MN2 drain paths), before $V_{m,out}$.
- *out* is the final output node, downstream of $V_{m,out}$. Voltage `V_out` and current `I_out` are recorded.
- *GND* (node 0) is the global ground reference. `I_gnd` is derived by Kirchhoff's current law as the negative sum of all other node currents.

Table 1 below shows the simulation results that will be captured as features in the dataset.

2.2. Simulation Methodology

The dataset was generated by simulating, using ngspice, the circuit testbench across different process models, process corners and skews, and transistor sizes. Each simulation represents a unique combination of process technology, operating conditions, device asymmetry, and circuit parameters. These variations are described below.

Process Models (9 variations) [11]. The simulation covers nine different semiconductor process technologies:

- *22nm HP:* Low threshold voltage, thin oxide for high performance 22nm node
- *22nm LP:* Higher threshold voltage for reduced leakage and low power 22nm node
- *32nm HP:* High performance 32nm node
- *32nm LP:* Low power 32nm node
- *45nm HP:* High performance 45nm node
- *45nm LP:* Low power 45nm node
- *65nm Bulk:* Traditional CMOS bulk process (neither HP nor LP)
- *90nm Bulk:* Mature bulk technology (neither HP nor LP)
- *130nm Bulk:* Legacy bulk process (neither HP nor LP)

PVT Corners (5 variations). Process, Voltage, Temperature combinations for comprehensive operating conditions:

- *Typical:* Nominal operating conditions
- *Slow:* Conservative: Low voltage, high temperature
- *Fast:* Aggressive: High voltage, low temperature
- *Hot:* High temperature stress testing

Feature	Source	Description
V_vdd	$V_{m,vdd}$, node vdd	Peak voltage at VDD rail
V_in	$V_{m,in}$, node in	Peak voltage at input
V_out	$V_{m,out}$, node out	Peak voltage at output
V_gnd	node 0	Ground reference (always 0)
V_target	$V_{m,tgt}$, node target	Peak voltage at inter-stage node
I_vdd	$V_{m,vdd}$ branch	Peak supply current
I_in	$V_{m,in}$ branch	Peak input current
I_out	$V_{m,out}$ branch	Peak output current
I_gnd	Kirchhoff (derived)	Ground return current
I_target	$V_{m,tgt}$ branch	**Peak inter-stage current (prediction target)**

Table 1: Simulation Results as Dataset Features

- *Cold:* Low temperature operation

Process Skew Corners (5 variations). Device asymmetry variations capturing realistic NMOS/PMOS device mismatches:

- *TT:* Balanced device performance
- *FF:* Both devices fast (high performance)
- *SS:* Both devices slow (low leakage)
- *FS:* Asymmetric: affects rise vs fall times
- *SF:* Asymmetric: affects fall vs rise times

Monte Carlo Device Parameters. For each combination above, the following parameters are randomly sampled:

- *WN1, WN2:* NMOS transistor widths (process-specific ranges)
- *WP1, WP2:* PMOS transistor widths (process-specific ranges)
- *L1, L2:* Channel lengths, 20nm-600nm (node dependent)
- *VDD:* Supply voltage, 0.7V-2.0V (node dependent)
- *TEMP:* Operating temperature, -20°C to 125°C
- *Example 22nm ranges:* W: 0.2μm - 8μm, L: 20nm - 200nm

2.3. Dataset 1

Dataset 1 was generated using only a single process model, 22nm_HP.pm. This was to determine whether the model could be accurate on a smaller feature size. This means that for every combination of device parameters, 25 (5 corners x 5 skews) simulations were run. Figure 4 shows a single instance of Dataset 1. The dataset contains 100,000 instances.

ID is the instance number. Design is the circuit. Even though it's the same for every instance, it makes sense to include this in the dataset because of future work will include multiple designs. Size and Option are the process. 22 is the minimum feature size in nanometers and HP means it's the High Performance version of the model. This is as opposed to LP, which stands for Low Power. PVT and Skew are simulation corners. PVT is Process, Voltage, and Temperature. The voltage is the supply voltage. Skew is how the transistors perform. SS means both N-type and P-type are slow. Skew can also be fast/fast, fast/slow, slow/fast, and typical/typical. The Ws and Ls are transistor sizes. For example, WN1 is the gate width of the N-type transistor in the first stage. Similarly for the others. It's typical for the N-type and P-type transistors to have the same gate length, so there are only two lengths. VDD is the supply voltage and Temp is the temperature. The rest are the voltages and current for the nodes, the last being the target current the model is going to predict.


```
{
  "ID": 0,
  "Design": "2inv",
  "Size": "22",
  "Option": "HP",
  "PVT": "typical",
  "Skew": "SS",
  "WN1": 3.987860998114745e-06,
  "WP1": 7.346739531762215e-06,
  "WN2": 3.0587266286604007e-06,
  "WP2": 3.038069119888764e-06,
  "L1": 2.762855924312858e-08,
  "L2": 1.228588215527502e-07,
  "VDD": 0.8906585278880905,
  "Temp": 40.103664373134855,
  "V_vdd": 0.8906628,
  "V_gnd": 0.0,
  "V_in": 0.8906585,
  "V_out": 1.205001,
  "V_target": 0.9205885,
  "I_vdd": 0.00193829,
  "I_gnd": -0.0032357769343999998,
  "I_in": 0.0009869179,
  "I_out": 0.0003105529,
  "I_target": 1.61344e-08
},
```


Figure 4: Dataset 1 in JSON Format

2.4. Dataset 2

Dataset 2 was generated using all 12 process models. This was to determine whether the model could be generalized to a larger feature size. This means that for every combination of device parameters, 25 (5 corners x 5 skews) simulations were run. Figure 5 shows a single instance of dataset 2. The dataset contains 112,500 instances.


```
"VTH0_N": 0.3782,
"TOX_N": 2.25e-09,
"TOXP_N": 1.6e-09,
"TOXM_N": 2.25e-09,
"U0_N": 0.05928,
"UC_N": 0.0,
"VSAT_N": 100370.0,
"XJ_N": 3.92e-08,
"NDEP_N": 1.54e+18,
"NF_N": 1.5,
"ETA0_N": 0.0092,
"K2_N": 0.01,
"DVT2_N": -0.032,
"VOFF_N": -0.13,
"PCLM_N": 0.06,
"RDSW_N": 200.0,
"CGSO_N": 2.4e-10,
"CGDO_N": 2.4e-10,
"VTH0_P": -0.321,
"TOX_P": 2.35e-09,
"TOXP_P": 1.6e-09,
"TOXM_P": 2.35e-09,
"U0_P": 0.00835,
"UC_P": -3e-11,
"VSAT_P": 70000.0,
"XJ_P": 3.92e-08,
"NDEP_P": 1.14e+18,
"NF_P": 1.5,
"ETA0_P": 0.0092,
"VOFF_P": -0.126,
"RDSW_P": 240.0,
"CGSO_P": 2.4e-10,
"CGDO_P": 2.4e-10,
```


Figure 5: Part of Dataset 2

The second dataset has all of the same features as the first dataset, plus some more. The additional features are parameters that change between process models. The parser automatically removes any parameter that doesn't change at all. Some parameters don't change much, so they may need to be pruned. Also, because some of the numbers are huge and time are minuscule, they may need to be normalized. The new features in Dataset 2 are below. Note that they have _N and _P versions. This is because the parameters differ for N-type and P-type transistors.

VTH0: Zero-bias threshold voltage ("on" voltage). **VOFF:** Adjusts the subthreshold current curve so simulation matches measured data. **TOX / TOXP / TOXM:** Gate dielectric thickness, which determines gate capacitance. **U0:** Controls carrier mobility, which determines current drive. **UC:** Mobility degradation coefficient. **VSAT:** Carrier velocity saturation limit. **XJ:** Junction depth of source/drain diffusion. **NDEP:** Channel doping concentration. **DVT2:** Second-order short-channel threshold roll-off parameter. **ETA0:** Controls how drain voltage lowers the threshold voltage. **K2:** Second-order body effect coefficient. **PCLM:** Controls channel length modulation. **RDSW:** Source/drain sheet resistance per width. **CGSO / CGDO:** Gate-to-source overlap capacitance. **NF:** Number of gate fingers.

3. Technical Approach

The technical approach was to implement a Graph Attention Network (GAT) for regression on circuit edge weights, where the circuit is represented as a graph with nodes corresponding to circuit components (such as transistors and interconnects) and edges representing electrical connections. Each node is assigned a feature vector encoding its physical and electrical properties. The GAT learns to incorporate information from neighboring nodes using attention mechanisms, allowing it to adaptively weight the influence of each neighbor based on both the source and target node features. The model is trained to predict the peak current for the relevant edge (I_target), enabling accurate current estimation without requiring full SPICE simulations.

3.1. Implementation

Our training algorithm was implemented in Python using PyTorch and the PyG library [1] for GATv2Conv layers. Experiment tracking is performed with Weights & Biases. The main training and experiment orchestration code is in `python/sweep.py`, with supporting data and model code in `python/dataset.py`, `python/gan.py`, and `python/graph.py` (see Appendix). The code repository can be found here: `https://github.com/YosemiteSamurai/CurrentPrediction`.

3.2. Model Architecture

To predict current values, regression on the graph's edge weights with a GATv2 network [3] are used. For this problem, the graphs have been designed such that our training is entirely supervised. The model consists of three stages of `GATv2Conv` layers [1]:

1. An input layer mapping node features to a 64-dimensional hidden space, using 4 attention heads whose outputs are averaged.

2. Three hidden layers of the same width and head configuration, each followed by an ELU activation.
3. An output layer with 4 heads whose outputs are *concatenated*, producing a 64-dimensional node embedding (16 dimensions per head).

These three stages work together as follows:

3.2.1 Input Layer

The input layer takes the raw node features, which may vary in dimension depending on the dataset, and projects them into a fixed 64-dimensional hidden space. This is accomplished using a GATv2Conv layer with 4 attention heads. The choice of 4 attention heads is a balance between model expressiveness and computational efficiency. Using multiple heads allows the model to attend to different aspects of the neighborhood structure simultaneously, capturing diverse patterns in the data. Empirically, 4 heads provided sufficient capacity for the circuit prediction task without incurring excessive computational cost or overfitting. Fewer heads (e.g., 3) may limit the model's ability to capture complex relationships, while more heads (e.g., 5 or more) increase memory and computation without a clear gain in performance for this dataset.

Each head computes its own attention-weighted aggregation of neighboring node features, and the outputs of the 4 heads are averaged. In the GATv2Conv implementation, each attention head produces an output vector of size 64. When using 4 heads with averaging, the outputs from all heads are summed and divided by 4, resulting in a single 64-dimensional vector per node. This is in contrast to concatenation, which would produce a 256-dimensional output (4 heads $\times$ 64 dimensions each). Averaging preserves the original hidden size, ensuring the output remains 64-dimensional, providing a consistent embedding size for all nodes regardless of the original feature dimension.

The hidden size of 64 is a common choice in graph neural networks, providing enough capacity to encode rich node representations while keeping the model tractable for large graphs. It is large enough to capture the relevant features and interactions in the circuit graphs, but not so large as to risk overfitting or slow down training significantly. This value was selected based on standard practice and preliminary experiments.

Consistent embedding size is crucial because subsequent layers in the network expect a fixed-size input for every node, regardless of the original feature vector's dimension. This uniformity allows the model to process graphs with nodes of varying feature sets (e.g., different circuit components) and ensures compatibility with batch processing and downstream layers. By projecting all node features into the same 64-dimensional space, the model can generalize across different node types and datasets, facilitating transfer learning and robust performance.

3.2.2 Hidden Layers

The next three layers are also GATv2Conv layers, each with 4 attention heads and a hidden size of 64. The choice of three hidden layers is a balance between model depth and the risk of overfitting or vanishing gradients. With each additional layer, the model can aggregate information from nodes that are further away in the graph, enabling it to capture more complex dependencies. The rationale having 4 attention heads is similar to the input layer: multiple heads allow the model to capture diverse patterns and relationships in the graph. Four heads provide a good trade-off between expressiveness and computational cost, as validated by experiments. Using the same number of heads across layers also simplifies the architecture and ensures consistent dimensionality.

After each layer, an Exponential Linear Unit (ELU) activation is applied to introduce nonlinearity. Nonlinearity is essential in neural networks to allow the model to learn complex, non-linear relationships in the data. Without nonlinear activations, the network would be limited to learning only linear transformations, regardless of its depth. The Exponential Linear Unit (ELU) is chosen because it helps mitigate the vanishing gradient problem, speeds up learning, and can produce negative outputs, which may be beneficial for representing certain circuit features. ELUs often lead to faster convergence and better generalization compared to ReLU or other activations in some graph neural network applications.

The hidden layers allow the model to iteratively refine node representations by aggregating information from increasingly distant neighbors in the graph. The attention mechanism enables the model to focus on the most relevant nodes for each prediction, which is especially important in circuit graphs where certain nodes (such as supply rails or transistors) play more critical roles than others.

While graph size can influence architectural choices, the decisions here are primarily based on standard practice in graph neural networks. The circuit graphs in this work are of moderate size, so three

layers are sufficient to capture relevant dependencies without excessive depth. The number of attention heads and the use of ELU are chosen for their general effectiveness rather than being tightly coupled to graph size. For much larger or more complex graphs, deeper networks or different configurations might be explored.

3.2.3 Output Layer

The final GATv2Conv layer also uses 4 attention heads, but instead of averaging their outputs, it concatenates them. The rationale for using 4 attention heads in the output layer is consistent with the earlier layers: multiple heads allow the model to capture different aspects of the node's neighborhood and learn a more nuanced representation. Four heads provide a good balance between model expressiveness and computational efficiency, as established in the input and hidden layers.

In the output layer, concatenation is used instead of averaging to preserve the unique information captured by each attention head. While averaging blends the outputs into a single vector (as in the input and hidden layers), concatenation retains the distinct features learned by each head, resulting in a richer and more expressive final embedding. This is particularly valuable for the final prediction, as it allows the decoder to leverage a broader set of features when estimating the target current. Concatenation increases the dimensionality (4 heads $\times$ 16 dimensions = 64), but this is intentional to maximize the information available for the final regression task.

This results in a 64-dimensional node embedding, with each head contributing 16 dimensions. The concatenated output provides a richer and more expressive representation, which is then used by the decoder to predict the target edge current. This design allows the model to capture a diverse set of features from different attention heads, improving its ability to generalize across different circuit configurations.

3.3. Graphical Circuit Models

Creating a graphical model of a circuit can be approached in several different ways. In physics problems, a circuit is typically translated to graph form by treating partitions of wire with equal potential as the nodes. For this case, however, it is preferable to encode circuit components as nodes to limit feature data to nodes and label data to edges, respectively. This allows the model to learn the relationship between component features and current flow, rather than relying on the model to infer the presence of components from patterns in the adjacency matrix. It also allows for more flexible modeling of complex circuit topologies, where multiple components may be connected in parallel or series, without needing to explicitly encode these configurations in the graph structure.

3.4. Feature Engineering and Loss Function

Feature engineering encompasses all transformations applied to both input features and target labels to make the data suitable for model training (see Appendix, `python/dataset.py`). For the input features, two main strategies are used: (1) Convert descriptive parameters to numerical encodings, using their implied ordinality. For example, Skew is converted to a two-dimensional parameter describing the corner placement, and Option is converted to a one-dimensional parameter encoding low, medium, or high power. The PVT parameters are dropped due to redundancy with other numerical parameters. (2) To address imbalance in feature scale, features are standardized across the entire dataset to a mean of zero and variance of one. This prevents cases of exploding gradient during training.

For the target labels (currents), label normalization is performed as a specialized form of feature engineering. The five branch currents in the dataset span approximately six orders of magnitude (8 pA to 46 μA). Training directly on raw current values with MAPE or MAE loss produced unstable gradients and poor convergence on small-magnitude currents. To address this, all current labels are normalized using a two-step process:

$$\hat{y} = \frac{\log_{10}(|I|) - \mu}{\sigma} \tag{2}$$

where μ and σ are computed from the flattened log-magnitudes of all current values across the full training set. This compresses the six-order dynamic range to a near-unit-scale output target. At evaluation time the prediction is inverse-transformed as $I = 10^{\hat{y}\,\sigma+\mu}$. With log-normalized labels, standard L1 loss (mean absolute error) becomes equivalent to a log-space relative error:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} |\hat{y}_i - y_i| \tag{3}$$

Critically, the loss is computed *only on I_target edges*. At inference time the model receives only transistor dimensions and process/PVT parameters

— no other branch currents are available. Masking the loss to I_target ensures the gradient signal reflects the actual prediction objective and avoids dilution from the five other edge types (I_vdd, I_gnd, I_in, I_out, and their junction duplicates).

3.5. Batching

In standard machine learning, all data samples (e.g., images) have the same shape. However, in graph learning, each graph can have a different number of nodes and edges, so their feature and adjacency matrices have different sizes. Because of this inconsistency, it's not possible to use standard batching techniques. The solution is to merge all of the graphs in a batch into a single, larger "supergraph" by concatenating all of the node feature matrices vertically, then placing each graph's adjacency matrix along the diagonal of a large, block-diagonal matrix (with zeroes elsewhere). This way, the batch is represented as one large graph, and the GAN can process all graphs in the batch together efficiently. [1] (Batching implementation: see Appendix, `sweep.py` and `graph.py`) At low batch sizes typical problems with stability were observed and large batch sizes led to a notable increase in measured error, so a batch size of 32 was chosen as a balance between stability and training speed.

3.6. Decoder

After the GATv2 encoder processes the graph, each node is represented by a 64-dimensional embedding vector $\mathbf{z}_i$ that encodes its physical, electrical, and topological context. The decoder's job is to predict the current for each edge (u, v) using these node embeddings. To do this, the decoder constructs a feature vector for each edge by combining information from both endpoint nodes in several ways:

- $\mathbf{z}_u$: Embedding of the source node (e.g., a transistor or interconnect)
- $\mathbf{z}_v$: Embedding of the target node
- $\mathbf{z}_u - \mathbf{z}_v$: Element-wise difference, capturing the relative relationship between the nodes
- $\mathbf{z}_u \odot \mathbf{z}_v$: Element-wise (Hadamard) product, capturing interactions between corresponding features

These four 64-dimensional vectors are concatenated to form a single 256-dimensional input vector for each edge:

$$[\mathbf{z}_u \,\|\, \mathbf{z}_v \,\|\, \mathbf{z}_u - \mathbf{z}_v \,\|\, \mathbf{z}_u \odot \mathbf{z}_v] \in \mathbb{R}^{256} \tag{4}$$

This design allows the decoder to leverage both the individual properties of each node and their pairwise relationships, which is important for accurately modeling current flow that depends on both local and contextual circuit features. The 256-dimensional edge feature vector is then passed through a two-layer multilayer perceptron (MLP):

- **First layer:** Fully connected, 256 → 64, followed by a ReLU activation. This layer learns nonlinear combinations of the edge features.
- **Second layer:** Fully connected, 64 → 1, producing a single scalar output $\hat{y}_{uv}$, which is the predicted log-normalized current for that edge.

Mathematically, the decoder can be written as:

$$\hat{y}_{uv} = \mathrm{MLP}([\mathbf{z}_u \,\|\, \mathbf{z}_v \,\|\, \mathbf{z}_u - \mathbf{z}_v \,\|\, \mathbf{z}_u \odot \mathbf{z}_v]) \tag{5}$$

This approach is well-suited for edge regression tasks in graph neural networks, as it allows the model to flexibly capture both additive and multiplicative interactions between node features, as well as their differences, which can be critical for predicting physical quantities like current that depend on the relationship between two circuit elements. The use of a two-layer MLP provides sufficient capacity to model complex nonlinear dependencies while remaining computationally efficient.

3.7. Optimization

The model is trained with the Adam optimizer at an initial learning rate of 1×10^{-4}. A `ReduceLROnPlateau` scheduler halves the learning rate after 8 consecutive epochs without improvement in test loss, with a minimum learning rate of 1×10^{-6}. Training runs for 100 epochs with a batch size of 32 graphs, on a 70/30 train/test split of Dataset 1 (70,000 training, 30,000 test samples). Training is performed on an NVIDIA RTX 8000 GPU via the OSU College of Engineering HPC cluster.

4. Experiment

The goal of our experiment is to directly compare two training regimes: one where the model is trained on Dataset 2 (multi-process, spanning nine manufacturing process technologies), and one where the model is trained on Dataset 1 (single process, 22 nm HP only). This comparison is designed to answer two key questions:

1. Can a GAN predict the current accurately enough to effectively replace a SPICE simulation, as shown in Figure 2?

2. If so, can a single model generalize across all processes without sacrificing accuracy, or does there need to be a model per process?

All experiments were tracked and visualized using Weights & Biases (wandb): `https://wandb.ai/yosemitesamurai/CurrentPrediction`.

4.1. Design

All experiments use the block-inverter graph model (`block_2inv`), 3 hidden GATv2 layers, hidden dimension 64, 4 attention heads, batch size 32, and initial learning rate 1×10^{-4}.

4.2. Evaluation

The primary evaluation metric is Mean Relative Error (MRE) on I_target in physical units (Amperes), computed on the held-out test set:

$$\mathrm{MRE} = \frac{1}{N} \sum_{i=1}^{N} \left| \frac{I_{\mathrm{pred},i} - I_{\mathrm{true},i}}{I_{\mathrm{true},i}} \right| \tag{6}$$

Predictions are inverse-transformed from log-normalized space before this calculation. MRE is evaluated **only on I_target edges** — the only current available at inference time and the quantity of engineering interest. We also report test L1 loss in log-normalized space as a stable training diagnostic.

4.3. Results

We compare the performance of the model when trained on Dataset 2 (multi-process) versus Dataset 1 (single process). The central question is whether a single model can generalize across all process technologies, or if accuracy is fundamentally limited by the need to distinguish between process clusters. The following results and figures summarize the training dynamics, error metrics, and learning rate schedules for both datasets side by side.

Figure is the training loss (L1) over epochs for both Dataset 1 (single process) and Dataset 2 (multi-process). Figure is the mean relative error (MRE) on I_target during training for both datasets. Figure is the learning rate schedule during training for both datasets. All three figures were tracked and visualized with wandb.

Figure 6: Training loss (L1) over epochs

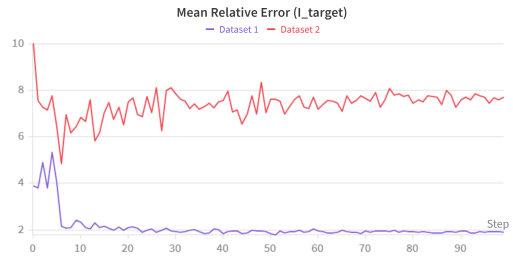

Figure 7: Mean relative error (MRE) on I_target

Figure 8: Learning rate schedule

4.3.1 Dataset Comparison

The single-process model achieves a $4\times$ reduction in MRE ($7.7\times \rightarrow 1.91\times$) despite having access to fewer than one-fifth of the distinct process configurations. This confirms that generality goal described in Section 4, a single model spanning all process nodes, cannot be achieved without sacrificing accuracy: the model's capacity is consumed by inter-process discrimination rather than intra-circuit current prediction. The per-process architecture described in Section 5.1.3 is therefore adopted as the primary path forward.

Training Set	Processes	Final Test L1	Final MRE
Dataset 2 (multi-process)	9	0.187	$7.7\times$
Dataset 1 (single process)	1	0.221	$\mathbf{1.91}\times$

Table 2: Head-to-head comparison of multi-process and single-process training. MRE is in physical Amperes on the held-out test set.

5. Conclusion

The answers to the questions posed in the Experiment section are: Yes and No.

Yes. For a single process (Dataset 1), the model achieves a mean relative error (MRE) of $1.91\times$ on the held-out test set, demonstrating that the GATv2 can predict inter-stage current with sufficient accuracy for practical use in design flows.

No. The multi-process model (Dataset 2) achieves an MRE of $7.7\times$, which is $4\times$ worse than the single-process model. This indicates that a single model spanning all process nodes cannot achieve the same accuracy as dedicated per-process models. The correct path forward is to train a separate model for each process technology.

This work demonstrates that a GATv2 graph neural network can be trained to predict the inter-stage peak current (I_target) of a cascaded CMOS inverter chain from transistor dimensions and process parameters alone, without any SPICE simulation at inference time. Two training regimes were evaluated: a multi-process model trained on Dataset 2 (nine process technologies, 112,500 simulations) and a single-process model trained on Dataset 1 (22 nm HP, 100,000 simulations).

The multi-process model (Dataset 2) converged to a test L1 loss of 0.187 and an I_target MRE of $7.7\times$ after 100 epochs, with the `ReduceLROnPlateau` scheduler halving the learning rate four times before stalling. This plateau is evidence of a capacity ceiling: a significant portion of the model's expressiveness is consumed distinguishing nine process-technology clusters rather than learning the intra-circuit current mapping.

Training on Dataset 1 removes the 33 BSIM4 process parameters from the feature vector entirely, reducing node feature dimensionality from $\sim$56 to $\sim$10 and eliminating the inter-process discrimination burden. The single-process model (Dataset 1) achieved a final MRE of $1.91\times$, a $4\times$ improvement over the multi-process model, with the learning rate halved only once across 100 epochs, indicating clean and stable convergence. The comparison in Section 4.3.1 confirms that the generality goal described in Section 4, a single model spanning all process nodes, cannot be achieved without a substantial accuracy cost. Accordingly, this work concludes that the correct path forward is one dedicated model per manufacturing process, trained on data generated solely from that process's simulation model.

5.1. Future Work

The following future work directions aim to improve the automation, generalizability, and privacy of the current prediction pipeline. Key goals include automating dataset/model generation, integrating feature engineering into data preprocessing, expanding to per-process and multi-design models, and developing privacy-preserving training methods. These steps will enable broader applicability, reproducibility, and secure deployment of the approach in real-world semiconductor design environments.

5.1.1 Integrated Pipeline

Currently, dataset generation (SPICE simulation via SLURM), parsing, and model training are separate manual steps. A single automated flow, triggered by supplying a process model file and a set of design netlists, would run simulations, parse results into the per-process dataset format, and launch training, producing a deployable model checkpoint and an inference endpoint with no manual intervention. This would lower the barrier to adding new process nodes or circuits.

5.1.2 Integrating Feature Engineering

Currently, feature engineering (such as categorical encoding and feature standardization) and label normalization (log-scaling and standardization) are performed as part of the model training pipeline. Moving these steps into the dataset generation process would improve reproducibility, simplify the training workflow, and ensure that all downstream users of the dataset apply the same transformations. This approach is common in large-scale machine learning projects, where preprocessing is tightly coupled to dataset versioning.

If feature engineering is performed during dataset generation, the dataset should include both the transformed features and the parameters used for standardization (e.g., means and variances for each feature). This ensures that any future model training or inference can apply the exact same transformations, avoiding data leakage or inconsistencies. Similarly, if label normalization (such as log-scaling and standardization) is moved to dataset generation, the normalization parameters (μ, σ) must be stored alongside the dataset and used for both training and inverse transformation at evaluation time.

Potential caveats include the need to carefully track and version the preprocessing parameters, especially if the dataset is updated or extended. For maximum flexibility, it may be beneficial to store both raw and processed versions of the data, or to provide scripts that can regenerate the processed dataset from raw simulation outputs using the same transformation logic. Overall, integrating feature engineering and label normalization into dataset generation is a recommended best practice for improving reproducibility and workflow efficiency.

5.1.3 Per-Process Models

This work validated the per-process training approach using the 22 nm HP process only. The natural next step is to generate datasets for each of the remaining eight process nodes and train a dedicated model for each, following the same pipeline. Comparing the final I_target MRE across all nine models will confirm whether the accuracy improvement observed for 22 nm HP generalises across process technologies and will identify any processes that may require additional data or architectural tuning.

5.1.4 Multi-Design Datasets

The current datasets contain only the two-inverter chain (2inv). A more robust evaluation requires simulation data from multiple circuit designs (e.g. ring oscillators, NAND/NOR gates, full adder stages). Expanding the dataset to cover several designs and retraining the per-process models will test whether the GATv2 graph representation generalizes across circuit topologies, which is a prerequisite for deployment in a real place-and-route flow.

5.1.5 Privacy-Preserving Training

The per-process model architecture maps naturally onto the real-world privacy boundary: a single model trained on data from one foundry's process model encodes only that foundry's IP. In commercial semiconductor design, process model files (`.pm`) are delivered under strict NDA. Three approaches are under consideration for hardening the training pipeline:

Split Encoder Architecture (most practical). The foundry trains a small private encoder locally that maps raw BSIM4 parameters to a compact fixed-size embedding (e.g. 16 floats). The foundry shares only the embedding vectors alongside simulation I/O pairs. The design house trains the GATv2 layers on `(W, L, VDD, Temp, Skew, process_embedding)` $\rightarrow$ I_target. The encoder weights remain at the foundry; neither the model weights nor the embeddings alone are sufficient to recover the original process parameters.

Federated Learning with Differential Privacy. The foundry runs simulations locally and shares only gradient updates, never raw data. Calibrated Gaussian noise is added to gradients before transmission (DP-SGD [2]), providing a formal privacy budget ε. The PyTorch Opacus library supports DP-SGD and would integrate directly with the existing training loop.

Secure Inference via Trusted Execution. Model weights are encrypted at rest and decrypted only inside a Trusted Execution Environment (Intel SGX or AMD SEV) at inference time, protecting both the foundry's process information encoded in the weights and the design house's model IP.

Acknowledgements

The author would like to thank Alexander La Chapelle and Jonathan Guzman for their contribution to the original version of this paper for our Deep Learning class Oregon State University. The author would also like to thank Dr. Fuxin Li, the course instruction, for his feedback, which directly led to these improvements. The original paper can be found here: https://web.engr.oregonstate.edu/~jonesm25/writing/Current%20Prediction%20in%20Analog%20Integrated%20Circuits.pdf

References

[1] Pyg documentation, 2026. [Online; accessed March 18, 2026]. URL: https://pytorch-geometric.readthedocs.io/en/latest/index.html.

[2] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 308–318, 2016. doi:10.1145/2976749.2978318.

[3] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022. URL: https://openreview.net/forum?id=F72ximsx7C1.

[4] Donghyeok Cho, Taekwang Jang, and SeongHwan Cho. A fully integrated 4:1 dc–dc converter with series transmission and parallel reception through electromagnetically coupled class-d lc oscillators. *IEEE Journal of Solid-State Circuits*, 60(10):3464–3473, Oct 2025. doi:10.1109/JSSC.2025.3566479.

[5] Qing Dong, Mahmut E. Sinangil, Burak Erbagci, Dar Sun, Win-San Khwa, Hung-Jen Liao, Yih Wang, and Jonathan Chang. 15.3 a 351tops/w and 372.4gops compute-in-memory sram macro in 7nm finfet cmos for machine-learning applications. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*, pages 242–244, Feb 2020. doi:10.1109/ISSCC19947.2020.9062985.

[6] Ziad El Sayed, Zeng Wang, Hana Selmani, Johann Knechtel, Ozgur Sinanoglu, and Lilas Alrahis. Graph neural networks for integrated circuit design, reliability, and security: Survey and tool. *ACM Comput. Surv.*, 58(4), October 2025. doi:10.1145/3769081.

[7] Luke R. Everson, Muqing Liu, Nakul Pande, and Chris H. Kim. An energy-efficient one-shot time-based neural network accelerator employing dynamic threshold error correction in 65 nm. *IEEE Journal of Solid-State Circuits*, 54(10):2777–2785, Oct 2019. doi:10.1109/JSSC.2019.2914361.

[8] Kathleen et al Feng. Amber: A 16-nm system-on-chip with a coarse- grained reconfigurable array for flexible acceleration of dense linear algebra. *IEEE Journal of Solid-State Circuits*, 59(3):947–959, March 2024. doi:10.1109/JSSC.2023.3313116.

[9] Himanshu Kaul, Mark A. Anders, Sanu K. Mathew, Gregory Chen, Sudhir K. Satpathy, Steven K. Hsu, Amit Agarwal, and Ram K. Krishnamurthy. 14.4 a 21.5m-query-vectors/s 3.37nj/vector reconfigurable k-nearest-neighbor accelerator with adaptive precision in 14nm tri-gate cmos. In *2016 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 260–261, Jan 2016. doi:10.1109/ISSCC.2016.7418006.

[10] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016. URL: http://arxiv.org/abs/1609.02907, arXiv:1609.02907.

[11] Microelectronics Research Center, University of Minnesota. Predictive technology model (ptm). https://mec.umn.edu/ptm/#h.v5g6eyuivo9x, n.d. Accessed: 2026-03-16.

[12] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chipdesign. *Nature*, 594(7862):207–212, Jun 2021. doi:10.1038/s41586-021-03544-w.

[13] Myeong-Jae et al Park. A 192-gb 12-high 896-gb/s hbm3 dram with a tsv auto-calibration scheme and machine-learning-based layout optimization. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 444–446, Feb 2022. doi:10.1109/ISSCC42614.2022.9731562.

[14] Myeong-Jae et al Park. A 192-gb 12-high 896-gb/s hbm3 dram with a tsv auto-calibration scheme and machine-learning-based layout optimization. *IEEE Journal of Solid-State Circuits*, 58(1):256–269, Jan 2023. doi:10.1109/JSSC.2022.3193354.

[15] Mahmut E. Sinangil, Yen-Ting Lin, Hung-Jen Liao, and Jonathan Chang. A 290-mv, 7-nm ultra-low-voltage one-port sram compiler design using a 12t write contention and read upset free bit-cell. *IEEE Journal of Solid-State Circuits*, 54(4):1152–1160, April 2019. doi:10.1109/JSSC.2019.2895236.

[16] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL: https://arxiv.org/abs/1710.10903.

[17] Chenyang Zhao, Jinbei Fang, Xiaoli Huang, Deyang Chen, Zhiwang Guo, Jingwen Jiang, Jiawei Wang, Jianguo Yang, Jun Han, Peng Zhou, Xiaoyong Xue, and Xiaoyang Zeng. A 28-nm 36 kb sram cim engine with 0.173 m2 4t1t cell and self-load-0 weight update for ai inference and training applications. *IEEE Journal of Solid-State Circuits*, 59(10):3277–3289, Oct 2024. doi:10.1109/JSSC.2024.3399615.

[18] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Graph neural networks: A review of methods and applications. *CoRR*, abs/1812.08434, 2018. URL: http://arxiv.org/abs/1812.08434, arXiv:1812.08434.

[19] Jonathan Zhou, Emir Ali Karahan, Sherif Ghozzy, Zheng Liu, Hossein Jalili, and Kaushik Sengupta. 25.3 ai-enabled design space discovery and end-to-end synthesis for rfics with reinforcement learning and inverse methods demonstrating mm-wave/sub-thz pas between 30 and 120ghz. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 68, pages 1–3, Feb 2025. doi:10.1109/ISSCC49661.2025.10904600.

Appendix

All code and netlists used in this work are included below for report completeness. The GitHub repository can be found here: `https://github.com/YosemiteSamurai/CurrentPrediction`.

Spice Netlists

2inv.sp

```
* ============================================
* Cascaded Inverter Testbench for ML Dataset
* All node currents and voltages measurable
* ============================================

.option post
.option nomod
.option accurate
.option method=gear

* --------- Subcircuits for current measurement ---------

.subckt PMOS_MEAS D G S B Dmeas 0 PARAMS: WP=1u L=100n
Vmeas_p D Dmeas 0
M1 Dmeas G S B pmos W={WP} L={L}
.ends PMOS_MEAS

.subckt NMOS_MEAS D G S B Dmeas 0 PARAMS: WN=1u L=100n
Vmeas_n D Dmeas 0
M1 Dmeas G S B nmos W={WN} L={L}
.ends NMOS_MEAS

.subckt C_MEAS N1 N2 N1meas 0 PARAMS: CVAL=1f
Vmeas_c N1 N1meas 0
C1 N1meas N2 {CVAL}
.ends C_MEAS

* --------- Driver Inverter ---------
XMP1 target_int in vdd vdd target_mp1 0 PMOS_MEAS PARAMS: WP={WP1} L={L1}
XMN1 target_int in 0 0 target_mn1 0 NMOS_MEAS PARAMS: WN={WN1} L={L1}
Vmeas_target target target_int 0V

* --------- Load Inverter ---------
XMP2 out_meas target vdd vdd out_mp2 0 PMOS_MEAS PARAMS: WP={WP2} L={L2}
XMN2 out_meas target 0 0 out_mn2 0 NMOS_MEAS PARAMS: WN={WN2} L={L2}
Vmeas_out out out_meas 0

* --------- Load Cap (optional realism) ---------
XCload out 0 out_cload 0 C_MEAS PARAMS: CVAL=5f

* --------- Analysis ---------
.tran 1p 5n

* --------- Model Files ---------
.include models/22nm_HP.pm

* --------- Output ---------
.control
run
print v(vdd) v(in) v(out) v(target)
print vmeas_vdd#branch vmeas_in#branch vmeas_out#branch vmeas_target#branch
print tran I(vmeas_vdd) I(vmeas_in) I(vmeas_out) I(vmeas_target)
.endc

* --------- Supplies ---------
VDD vdd_src 0 {VDD}
Rsupply vdd_meas vdd_src 0.01
Vmeas_vdd vdd vdd_meas 0

* --------- Input ---------
Vin in_src 0 PULSE(0 {VDD} 0 20p 20p 1n 2n)
Rin in_meas in_src 0.01
Vmeas_in in in_meas 0

* --------- Ground connection (forced to 0)
* gnd is now directly connected to SPICE ground
* Remove Rgnd, connect gnd to 0 with a wire
* In SPICE, just use the same node name for both:
* All references to 'gnd' are now node 0
```

template.sp

```
* Auto-generated

.param WN1={WN1}
.param WP1={WP1}
.param WN2={WN2}
.param WP2={WP2}
.param L1={L1}
.param L2={L2}

.param VDD={VDD}
.param TEMP={TEMP}

.include "{model}"
.include "{pvt_corner}"
.include "{skew_corner}"
```

```
* All measurements are defined in 2inv.sp
.include "2inv.sp"
```

Python Code

dataset.py

```
# ==========================================================================
# dataset.py -- Dataset, Loss Functions, and Training Loop
#
# Contains everything needed to load data and train the GAN (GATv2), separated from
# the sweep/entry-point configuration in sweep.py.
#
# Data loading & preprocessing:
#   - Loads a JSON dataset from the local dataset/ directory
#   - Encodes categoricals: Skew (S/T/F -> 0/1/2), Option (LP/bulk/HP -> 0/1/2)
#   - StandardScaler-normalizes input features; labels are log10 + standardized
#
# circuit_dataset: PyTorch Dataset that converts each data row into a
#   (edges, X) graph tensor pair by dispatching to the appropriate model
#   encoder in models.py.
#
# Custom loss functions:
#   - NMAELoss: Normalized MAE weighted by inverse of each label's mean
#   - LogL1Loss: L1 loss computed in log-space
#   - MAPELoss / LogMAPELoss: Mean absolute percentage error (active default)
#
# Training loop:
#   - train: Single epoch; encodes -> decodes -> backpropagates per batch
#   - test: Evaluates MAPE, mean relative error, and min/max relative error
# ==========================================================================

import json
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import Dataset, DataLoader
import torch
import torch.nn as nn
import torch.nn.functional as F
import models
from graph import Graph, batch_graph

print("[dataset] imports complete", flush=True)

_dataset_dir = os.path.join(os.path.dirname(__file__), "..", "dataset")

_process_datasets = sorted(

    os.path.join(_dataset_dir, f) for f in os.listdir(_dataset_dir)
    if f.endswith("_dataset.json")

)

if _process_datasets:

    DATA_FILE = _process_datasets[0]
    print(f"[dataset] auto-selected dataset: {os.path.basename(DATA_FILE)}", flush=True)

else:

    DATA_FILE = os.path.join(_dataset_dir, "dataset.json")
    print(f"[dataset] falling back to dataset.json", flush=True)

print("[dataset] loading JSON...", flush=True)

with open(DATA_FILE, "r") as f:
    data = json.load(f)

print(f"[dataset] loaded {len(data)} rows", flush=True)

data_frame = pd.DataFrame(data)
data_frame = data_frame.drop(columns=["ID", "PVT"])
design_col = data_frame['Design']
data_frame = data_frame.drop(columns=["Design"])

SKEW_CODES = {'S': 0,
              'T': 1,
              'F': 2}

data_frame['SkewL'] = data_frame['Skew'].apply(lambda x: SKEW_CODES[x[0]])
data_frame['SkewR'] = data_frame['Skew'].apply(lambda x: SKEW_CODES[x[1]])
data_frame = data_frame.drop(columns=["Skew"])

OPTION_CODES = {'LP': 0,
                'bulk': 1,
                'HP': 2}

data_frame['Option'] = data_frame['Option'].apply(lambda x: OPTION_CODES[x])
label_df = data_frame.loc[:, "I_vdd":"I_target":1]
print("[dataset] normalizing labels...", flush=True)

label_values = np.maximum(np.abs(label_df.values), 1e-30)
all_log_labels = np.log10(label_values.flatten())
label_log_mean = float(all_log_labels.mean())
label_log_std = float(all_log_labels.std())

log_label_df = pd.DataFrame(
    (np.log10(label_values) - label_log_mean) / label_log_std,
    columns=label_df.columns,
```

```
    index=label_df.index
)

scaler = StandardScaler()  # saved in checkpoint for inference feature scaling
scaled_data = scaler.fit_transform(data_frame)
data_frame = pd.DataFrame(scaled_data, columns=data_frame.columns)
data_frame['Design'] = design_col
data_frame.loc[:, "I_vdd":"I_target":1] = log_label_df

print("==================Data loaded successfully===============")
print("head")
print(data_frame.head())
print("infomation")
print(data_frame.info())
print("size of DF")
print(data_frame.shape)

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
print(f"[dataset] device: {device}", flush=True)

class circuit_dataset(Dataset):

    def __init__(self, data_frame, config):

        self.df = data_frame.reset_index(drop=True)
        self.config = config

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):

        row = self.df.iloc[idx]
        design = row['Design']
        model = getattr(models, self.config.model + '_' + design)
        return model(row, design)

class NMAELoss(nn.L1Loss):

    def __init__(self, reduction: str = "mean"):

        super().__init__()
        label_mean = label_df.mean().values
        self.norm = [1/label_mean[0]]*2 + [1/label_mean[1]]*2 + [1/label_mean[2]]*2 + [1/label_mean[3]]*2 + [1/label_mean[4]]*4

    def forward(self, y_pred, y_true):

        norm = torch.tensor(self.norm * int(y_pred.shape[0] / len(self.norm))).to(device)
        return F.l1_loss(torch.mul(y_pred, norm), torch.mul(y_true, norm))

class LogL1Loss(nn.L1Loss):

    def __init__(self, reduction: str = "mean"):
        super().__init__()

    def forward(self, y_pred, y_true):
        return F.l1_loss(torch.log10(y_pred), torch.log10(y_true))

def MAPELoss(y_pred, y_true):

    epsilon = 1e-8  # To avoid division by zero

    return torch.mean(torch.abs((y_true - y_pred) / (y_true + epsilon)))

def LogMAPELoss(y_pred, y_true):
    return MAPELoss(torch.log(y_pred), torch.log(y_true))

criterion = nn.L1Loss()

def train(gcn, optimizer, trainloader, config):

    total_loss = 0
    batches = 0

    for batch in trainloader:

        graph = batch_graph(batch, config)

        A = graph.A.to(device)
        y = graph.y.to(device)
        X = graph.X.to(device)

        optimizer.zero_grad()
        z = gcn.encode(X, A)
        out = gcn.decode(z, A).view(-1)
        n_edges = y.shape[0]
        mask = torch.zeros(n_edges, dtype=torch.bool, device=device)
        mask[config.target_edge_idx::config.edges_per_graph] = True
        loss = criterion(out[mask], y[mask])
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
        batches += 1

    return gcn, optimizer, total_loss / batches

def test(gcn, testloader, config):

    total_loss = 0
    batches = 0
    predictions = 0
    error = 0
    max_error = 0
    min_error = float('inf')
```

```
    with torch.no_grad():

        for batch in testloader:

            graph = batch_graph(batch, config)

            A = graph.A.to(device)
            y = graph.y.to(device)
            X = graph.X.to(device)

            z = gcn.encode(X, A)
            out = gcn.decode(z, A).view(-1)
            n_edges = y.shape[0]
            mask = torch.zeros(n_edges, dtype=torch.bool, device=device)
            mask[config.target_edge_idx::config.edges_per_graph] = True
            loss = criterion(out[mask], y[mask])
            total_loss += loss.item()
            batches += 1
            y_pred_phys = 10 ** (out[mask].cpu() * label_log_std + label_log_mean)
            y_true_phys = 10 ** (y[mask].cpu() * label_log_std + label_log_mean)
            error_tensor = torch.abs((y_pred_phys - y_true_phys) / (y_true_phys + 1e-30))

            if error_tensor.max() > max_error:
                max_error = error_tensor.max()

            if error_tensor.min() < min_error:
                min_error = error_tensor.min()

            error += error_tensor.sum()
            predictions += len(y_pred_phys)

    return total_loss / batches, error / predictions, max_error, min_error
```

extract_model_params.py

```
# ==========================================================================
# extract_model_params.py -- Model Parameter Extractor for SPICE Dataset
#
# Extracts key physical parameters from .pm model files for use in dataset generation and feature engineering.
#
# Main function:
#   - extract_model_params(model_file): Parses a SPICE model file and returns a dictionary of extracted NMOS and PMOS parameters.
# ==========================================================================

import re
import os

MODELS_DIR = "models"  # Update if models/ is moved

def extract_model_params(model_file):

    if not os.path.exists(model_file):
        return {}

    try:

        with open(model_file, 'r') as f:
            content = f.read()

        params = {}
        nmos_match = re.search(r'\.model\s+nmos.*?(?=\.model\s+pmos|$)', content, re.DOTALL | re.IGNORECASE)
        pmos_match = re.search(r'\.model\s+pmos.*?$', content, re.DOTALL | re.IGNORECASE)

        if nmos_match:

            nmos_section = nmos_match.group(0)
            params.update(extract_device_params(nmos_section, 'N'))

        if pmos_match:

            pmos_section = pmos_match.group(0)
            params.update(extract_device_params(pmos_section, 'P'))

        return params

    except Exception as e:

        print(f"Warning: Could not extract parameters from {model_file}: {e}")

        return {}

def extract_device_params(section, device_type):

    params = {}

    param_patterns = {
        f'VTH0_{device_type}': r'vth0\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'TOX_{device_type}': r'toxe\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'TOXP_{device_type}': r'toxp\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'TOXM_{device_type}': r'toxm\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'U0_{device_type}': r'u0\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'UA_{device_type}': r'ua\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'UB_{device_type}': r'ub\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'UC_{device_type}': r'uc\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'UA1_{device_type}': r'ua1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'UB1_{device_type}': r'ub1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'UC1_{device_type}': r'uc1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'VSAT_{device_type}': r'vsat\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'XJ_{device_type}': r'xj\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'NDEP_{device_type}': r'ndep\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'NSD_{device_type}': r'nsd\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'NF_{device_type}': r'nfactor\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
```

```
        f'ETA0_{device_type}': r'eta0\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'ETAB_{device_type}': r'etab\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'VFB_{device_type}': r'vfb\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'K1_{device_type}': r'k1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'K2_{device_type}': r'k2\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'K3_{device_type}': r'k3\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'K3B_{device_type}': r'k3b\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'DVT0_{device_type}': r'dvt0\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'DVT1_{device_type}': r'dvt1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'DVT2_{device_type}': r'dvt2\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'DSUB_{device_type}': r'dsub\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'MINV_{device_type}': r'minv\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'VOFF_{device_type}': r'voff\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'PCLM_{device_type}': r'pclm\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'PDIBLC1_{device_type}': r'pdiblc1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'PDIBLC2_{device_type}': r'pdiblc2\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'DROUT_{device_type}': r'drout\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'RDSW_{device_type}': r'rdsw\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'RSH_{device_type}': r'rsh\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'CGSO_{device_type}': r'cgso\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'CGDO_{device_type}': r'cgdo\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'CGBO_{device_type}': r'cgbo\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'ALPHA0_{device_type}': r'alpha0\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'ALPHA1_{device_type}': r'alpha1\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'BETA0_{device_type}': r'beta0\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'AGIDL_{device_type}': r'agidl\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'BGIDL_{device_type}': r'bgidl\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'CGIDL_{device_type}': r'cgidl\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
        f'EGIDL_{device_type}': r'egidl\s*=\s*([-+]?[0-9]*\.?[0-9]+(?:[eE][-+]?[0-9]+)?)',
    }

    for param_name, pattern in param_patterns.items():

        match = re.search(pattern, section, re.IGNORECASE)

        if match:

            try:
                params[param_name] = float(match.group(1))

            except ValueError:
                params[param_name] = None

        else:
            params[param_name] = None

    return params

def test_extraction():

    models_dir = "models"

    if not os.path.exists(models_dir):

        print("Models directory not found!")

        return

    for filename in os.listdir(models_dir):

        if filename.endswith('.pm'):

            model_path = os.path.join(models_dir, filename)
            print(f"\n=== {filename} ===")
            params = extract_model_params(model_path)

            for param, value in params.items():

                if value is not None:
                    print(f"{param}: {value:.3e}")

                else:
                    print(f"{param}: Not found")

if __name__ == "__main__":
    test_extraction()
```

gan.py

```
# ==============================================================================
# gan.py -- GATv2 Model Architecture
#
# Defines the GAN class using PyTorch Geometric's GATv2Conv layers.
# GATv2Conv learns a separate attention weight per neighbor per node, making
# it significantly more expressive than GCNConv for heterogeneous graphs like
# circuits where node roles (VDD, transistor, GND) differ greatly.
#
# encode(x, edge_index): Passes node features through a GATv2Conv input layer,
#   ELU activations, N optional hidden GAT layers, and a final GAT layer to
#   produce node embeddings. Hidden layers use multi-head attention with heads
#   averaged; the output layer concatenates heads to produce richer embeddings.
#
# decode(z, edge_label_index): Predicts each edge's current value using a
#   two-layer MLP over the concatenation of the two endpoint embeddings along
#   with their element-wise difference and product. This gives the decoder
#   explicit signal about the directional relationship between endpoints.
#
# Depth is controlled by extra_layers; heads controls multi-head attention.
# ==============================================================================

import torch
```

```python
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import GATv2Conv

class GAN(torch.nn.Module):

    def __init__(self, in_channels, hidden_channels, out_channels, extra_layers, heads=4):

        super(GAN, self).__init__()
        self.conv0 = GATv2Conv(in_channels, hidden_channels, heads=heads, concat=False)
        extra = []

        for _ in range(extra_layers):
            extra.append(GATv2Conv(hidden_channels, hidden_channels, heads=heads, concat=False))

        self.extra = nn.ModuleList(extra)
        self.convF = GATv2Conv(hidden_channels, out_channels, heads=heads, concat=True)
        embedding_dim = out_channels * heads
        mlp_input_dim = embedding_dim * 4

        self.mlp = nn.Sequential(

            nn.Linear(mlp_input_dim, hidden_channels),
            nn.ReLU(),
            nn.Linear(hidden_channels, 1)

        )

    def encode(self, x, edge_index):

        x = self.conv0(x, edge_index)
        x = F.elu(x)

        for layer in self.extra:

            x = layer(x, edge_index)
            x = F.elu(x)

        x = self.convF(x, edge_index)

        return x

    def decode(self, z, edge_label_index):

        z_u = z[edge_label_index[0]]
        z_v = z[edge_label_index[1]]
        edge_features = torch.cat([z_u, z_v, z_u - z_v, z_u * z_v], dim=-1)

        return self.mlp(edge_features).squeeze(-1)
```

graph.py

```python
# ==========================================================================
# graph.py -- Graph Data Structure
#
# Provides a Graph class that holds the circuit graph representation:
#   - A: COO-format sparse edge index (2xE tensor of source/dest node indices)
#   - y: Edge weight labels (simulated branch current values)
#   - X: Node feature matrix
#
# Key methods:
#   - add_edge: Appends a directed edge with its current label.
#   - merge(graph): Combines two graphs into one by offsetting the second
#     graph's node indices by self.num_nodes, enabling batch training.
#   - batch_graph(batch, config): Converts a raw DataLoader batch into a single
#     merged Graph object -- one graph per sample, all concatenated for
#     parallel GPU processing.
# ==========================================================================

import torch

class Graph():

    def __init__(self, edges, features, config):

        self.A = [[], []]
        self.A_blank = [[], []]
        self.y = []

        for edge in edges:
            self.add_edge(edge[0], edge[1], edge[2])

        self.A = torch.tensor(self.A)
        self.y = torch.tensor(self.y)
        self.X = features

        self.num_nodes = self.X.shape[0]

    def add_edge(self, start, end, weight):

        self.A[0].append(int(start))
        self.A[1].append(int(end))
        self.y.append(weight)

    def merge(self, graph):

        self.A = torch.cat((self.A, graph.A + self.num_nodes), dim=1)
        self.y = torch.cat((self.y, graph.y), dim=0)
        self.X = torch.cat((self.X, graph.X), dim=0)
        self.num_nodes += graph.num_nodes
```

```
def batch_graph(batch, config):

    graphs = []

    for i in range(len(batch[0])):
        edges = batch[0][i]
        features = batch[1][i]
        graphs.append(Graph(edges, features, config))

    batch_graph = graphs[0]
    for graph in graphs[1:]:
        batch_graph.merge(graph)

    return batch_graph
```

models.py

```
# ==========================================================================
# models.py -- Circuit-to-Graph Encoders
#
# Translates raw SPICE simulation data rows into (edges, X) graph tensors
# for specific circuit topologies. Implements variants for the 2-inverter chain (2inv):
#   - block_2inv (6 nodes): Coarse-grained model grouping PMOS+NMOS of each inverter into a
#     single transistor node (M1, M2). Topology: Vi -> M1 -> M2 -> Vo, with VDD and GND rails.
#   - split_2inv (8 nodes): Fine-grained model separating each inverter into individual P/N
#     transistor nodes (P1/N1, P2/N2), with extra edges to handle duplicated currents at junctions.
#
# Both models build node feature matrix X containing:
#   - Transistor dimensions: W, L per device
#   - SPICE model parameters: 15 PMOS params (VTH0, TOX, U0, ...) and 18 NMOS params
#   - Global conditions: temperature, process skew (L/R), technology size, device option (LP/HP/bulk)
#
# Edge weights are the simulated branch currents (I_vdd, I_in, I_out, I_gnd, I_target).
# ==========================================================================

import numpy as np
import torch

def _f(data, key):

    return float(data[key]) if key in data.index else 0.0

def block_2inv(data, design):

    NODE_MAP = {
      "Vi": 0,
      "M1": 1,
      "VDD": 2,
      "M2": 3,
      "Vo": 4,
      "GND": 5}

    NUM_NODES = len(NODE_MAP)

    Vi = NODE_MAP["Vi"]
    M1 = NODE_MAP["M1"]
    VDD = NODE_MAP["VDD"]
    M2 = NODE_MAP["M2"]
    Vo = NODE_MAP["Vo"]
    GND = NODE_MAP["GND"]

    I_t = data['I_target']
    I_in = data['I_in']
    I_out = data['I_out']
    I_vdd = data['I_vdd']
    I_gnd = data['I_gnd']

    edges = [(VDD, M1, I_vdd),
             (VDD, M2, I_vdd), # duplicated over junction
             (Vi, M1, I_in),
             (M1, M2, I_t),
             (M1, GND, I_gnd),
             (M2, GND, I_gnd), # duplicated over junction
             (M2, Vo, I_out)
            ]

    temp = data['Temp']
    skewl = data['SkewL']
    skewr = data['SkewR']
    size = _f(data, 'Size')
    option = _f(data, 'Option')
    general = [temp, skewl, skewr, size, option]
    pFields = ['VTH0', 'TOX', 'TOXP', 'TOXM', 'U0', 'UC', 'VSAT',
               'XJ', 'NDEP', 'NF', 'ETA0', 'VOFF', 'RDSW', 'CGSO',
               'CGDO']
    nFields = pFields + ['PCLM', 'K2', 'DVT2']

    NUM_SPECIFIC = 4 + len(pFields) + len(nFields)
    x = [0]*NUM_SPECIFIC + general
    X = np.repeat([x], NUM_NODES, axis=0)

    X[M1, 0] = data['WP1']
    X[M1, 1] = data['WN1']
    X[M1, 2] = data['L1']
    X[M2, 0] = data['WP2']
    X[M2, 1] = data['WN2']
    X[M2, 2] = data['L2']
    X[VDD, 3] = data['VDD']

    for i in range(len(pFields)):

        X[M1, 3 + i] = _f(data, pFields[i] + '_P')
```

```
        X[M2, 3 + i] = _f(data, pFields[i] + '_P')

    for i in range(len(nFields)):

        X[M1, 3 + len(pFields) + i] = _f(data, nFields[i] + '_N')
        X[M2, 3 + len(pFields) + i] = _f(data, nFields[i] + '_N')

    edges = torch.tensor(edges).to(torch.float)
    X = torch.tensor(X).to(torch.float)

    return edges, X

def split_2inv(data, design):

    NODE_MAP = {
      "Vi": 0,
      "N1": 1,
      "P1": 2,
      "VDD": 3,
      "N2": 4,
      "P2": 5,
      "Vo": 6,
      "GND": 7
    }

    NUM_NODES = len(NODE_MAP)

    Vi = NODE_MAP["Vi"]
    N1 = NODE_MAP["N1"]
    P1 = NODE_MAP["P1"]
    VDD = NODE_MAP["VDD"]
    N2 = NODE_MAP["N2"]
    P2 = NODE_MAP["P2"]
    Vo = NODE_MAP["Vo"]
    GND = NODE_MAP["GND"]

    I_t = data['I_target']
    I_in = data['I_in']
    I_out = data['I_out']
    I_vdd = data['I_vdd']
    I_gnd = data['I_gnd']

    edges =[(VDD, P1, I_vdd),
            (VDD, P2, I_vdd),
            (N1, GND, I_gnd),
            (N2, GND, I_gnd),
            (Vi, P1, I_in),
            (Vi, N1, I_in),
            (P2, Vo, I_out),
            (N2, Vo, I_out),
            (P1, P2, I_t),
            (P1, N2, I_t),
            (N1, P2, I_t),
            (N1, N2, I_t),
            ]

    temp = data['Temp']
    skewl = data['SkewL']
    skewr = data['SkewR']
    size = _f(data, 'Size')
    option = _f(data, 'Option')
    general = [temp, skewl, skewr, size, option]
    pFields = ['VTH0', 'TOX', 'TOXP', 'TOXM', 'U0', 'UC', 'VSAT',
               'XJ', 'NDEP', 'NF', 'ETA0', 'VOFF', 'RDSW', 'CGSO',
               'CGDO']
    nFields = pFields + ['PCLM', 'K2', 'DVT2']

    NUM_SPECIFIC = 3 + max(len(pFields), len(nFields))
    x = [0]*NUM_SPECIFIC + general
    X = np.repeat([x], NUM_NODES, axis=0)

    X[P1, 0] = data['WP1']
    X[P1, 1] = data['L1']
    X[N1, 0] = data['WN1']
    X[N1, 1] = data['L1']
    X[P2, 0] = data['WP2']
    X[P2, 1] = data['L2']
    X[N2, 0] = data['WN2']
    X[N2, 1] = data['L2']

    for i in range(len(pFields)):

        X[P1, 3 + i] = _f(data, pFields[i] + '_P')
        X[P2, 3 + i] = _f(data, pFields[i] + '_P')

    for i in range(len(nFields)):

        X[N1, 3 + i] = _f(data, nFields[i] + '_N')
        X[N2, 3 + i] = _f(data, nFields[i] + '_N')

    X[VDD, 2] = data['VDD']
    edges = torch.tensor(edges).to(torch.float)
    X = torch.tensor(X.astype(float)).to(torch.float)

    return edges, X
```

parse_results.py

```
# ==============================================================================
# parse_results.py -- Results Parser for SPICE Dataset
#
# Extracts PEAK SUPPLY CURRENT from simulation logs for GNN training.
```

```
# Target: Predict peak supply current from circuit parameters (no simulation needed).
#
# Main function:
#   - parse_spice_log(log_file): Parses a SPICE log file and extracts peak supply current and other metrics for GNN training.
#
# Usage:
#   Used during dataset generation to extract target values for supervised learning.
# ==========================================================================

import os
import json
import re
import numpy as np
import pandas as pd
from extract_model_params import extract_model_params

def parse_spice_log(log_file):

    try:

        with open(log_file, 'r') as f:
            content = f.read()

        stripped = content.lstrip()

        if stripped.startswith('{'):
            return json.loads(stripped)

        results = {
            'Target_Current': None,  # THE target for GNN prediction (renamed)
            'simulation_success': False,
            'error_message': None,
            'Node_Max_Voltage': {},
            'Node_Peak_Current': {}
        }

        if 'run simulation(s) aborted' in content or 'Fatal error' in content:

            error_match = re.search(r'Fatal error: (.+)', content)
            results['error_message'] = error_match.group(1) if error_match else 'Simulation failed'

            return results

        lines = content.split('\n')
        node_voltages = {}
        node_currents = {}

        def trunc15(s):
            return s[:15]

        node_current_vectors = {

            'vdd': trunc15('vmeas_vdd#branch'),
            'in': trunc15('vmeas_in#branch'),
            'out': trunc15('vmeas_out#branch'),
            'target': 'vmeas_target#br',

        }

        for line in lines:

            if re.match(r'^[a-zA-Z0-9_.#()]+\s+[-+eE0-9.]+$', line.strip()):

                parts = line.split()

                if len(parts) == 2:
                    name, val = parts

                    try:

                        fval = float(val)

                        if name.startswith('vmeas_') and name.endswith('#branch'):
                            node_currents.setdefault(name, []).append(abs(fval))

                        elif name.startswith('v(') and name.endswith(')'):

                            node = name[2:-1]
                            node_voltages.setdefault(node, []).append(fval)

                        elif name in node_current_vectors.values():
                            node_currents.setdefault(name, []).append(abs(fval))

                        elif name in ['vdd', 'in', 'out', 'target']:
                            node_voltages.setdefault(name, []).append(fval)

                    except Exception:
                        pass

        i = 0

        while i < len(lines):

            line = lines[i].strip()

            if line.startswith('Index') and 'time' in line:

                header = line
                m = re.findall(r'(v\([^)]*\)|[a-zA-Z0-9_#]+)', header)

                if m:

                    vars = [v[:15] for v in m[2:]]  # Skip Index, time
                    i += 2
```

```
            while i < len(lines):

                data_line = lines[i].strip()

                if (not data_line or data_line.startswith('-') or 'Index' in data_line or 'Total analysis' in data_line):
                    break

                try:

                    if '\t' in data_line:
                        parts = data_line.split('\t')

                    else:
                        parts = re.split(r'\s+', data_line)

                    for idx, var in enumerate(vars):

                        if len(parts) > idx+2:
                            val = float(parts[idx+2])

                            if var.startswith('v('):
                                node = var[2:-1]
                                node_voltages.setdefault(node, []).append(val)

                            elif var.endswith('#branc') or var.endswith('#branch'):
                                node_currents.setdefault(var, []).append(abs(val))

                except Exception:
                    pass

                i += 1
    i += 1


for node in ['vdd', 'in', 'out', 'target']:
    vlist = node_voltages.get(node, [])

    if vlist:
        results['Node_Max_Voltage'][node] = float(np.max(vlist))

node_peak_currents = {}

for node, vec in node_current_vectors.items():

    ilist = node_currents.get(vec, [])

    if not ilist:
        ilist = node_currents.get(vec + 'h', [])  # e.g., vmeas_vdd#branch

    if ilist:
        node_peak_currents[node] = float(np.max(ilist))

if 'target' not in node_peak_currents:

    for k in node_currents:

        if k.startswith('vmeas_target#br') and node_currents[k]:

            node_peak_currents['target'] = float(np.max(node_currents[k]))

            break

if node_peak_currents:
    node_peak_currents['gnd'] = -sum(node_peak_currents.get(n, 0.0) for n in ['vdd', 'in', 'out', 'target'])

results['Node_Peak_NodeCurrent'] = node_peak_currents

meas_pattern = re.compile(r'(I_VDD_MAX|I_Vin_MAX)\s*=\s*([-+]?\d*\.\d+e[+-]?\d+|[-+]?\d+\.\d+|[-+]?\d+)', re.IGNORECASE)

for line in lines:

    m = meas_pattern.search(line)

    if m:

        key = m.group(1).upper()
        val = float(m.group(2))

        if key == 'I_VDD_MAX':

            results['Target_Current'] = val
            results['Node_Peak_Current']['VDD'] = val
            results['simulation_success'] = True

        elif key == 'I_VIN_MAX':
            results['Node_Peak_Current']['VIN'] = val

if not results['simulation_success']:

    found_any = False

    for node, ilist in node_currents.items():

        if ilist:

            results['Node_Peak_Current'][node] = float(np.max(ilist))
            found_any = True

    if found_any:
        results['simulation_success'] = True

    if 'VDD' in node_currents and node_currents['VDD']:
        results['Target_Current'] = float(np.max(node_currents['VDD']))

return results
```

```
    except Exception as e:

        return {

            'target_current_peak': None,
            'target_current_avg': None,
            'target_current_rms': None,
            'simulation_success': False,
            'error_message': str(e)

        }

def create_dataset():

    import glob

    script_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(script_dir, '..'))
    results_dir = os.path.join(project_root, 'results')
    metadata = []
    meta_files = glob.glob(os.path.join(results_dir, 'metadata_*.json'))

    if not meta_files:
        raise FileNotFoundError(f"No metadata_*.json files found in {results_dir}. Run the simulations first!")

    for meta_path in meta_files:

        with open(meta_path, 'r') as f:

            try:

                meta = json.load(f)

                if isinstance(meta, list):
                    metadata.extend(meta)

                else:
                    metadata.append(meta)

            except Exception as e:
                print(f"Warning: Could not load {meta_path}: {e}")

    dataset = []
    process_tag = ''

    print(f"Processing {len(metadata)} simulation results...")

    for i, entry in enumerate(metadata):

        model_params = extract_model_params(entry['model'])
        model_name = os.path.basename(entry['model']).replace('.pm', '')
        pvt_name = os.path.basename(entry['pvt_corner']).replace('.sp', '')
        skew_name = os.path.basename(entry['skew_corner']).replace('skew_', '').replace('.sp', '')

        if '_' in model_name:

            feature_size_str, option = model_name.split('_', 1)
            feature_size = feature_size_str.replace('nm', '')

        else:

            feature_size = model_name.replace('nm', '')
            option = ''

        if not process_tag:

            process_tag = f"{feature_size}{option}"

        design_name = entry.get('design')

        if not design_name:
            design_name = os.environ.get('SPICE_DESIGN_NAME')

        if not design_name:
            design_name = os.environ.get('DESIGN')

        if not design_name:
            design_name = '2inv'

        features = {

            'ID': entry['run'],
            'Design': design_name,
            'PVT': pvt_name,
            'Skew': skew_name,
            'WN1': entry['params']['WN1'],
            'WP1': entry['params']['WP1'],
            'WN2': entry['params']['WN2'],
            'WP2': entry['params']['WP2'],
            'L1': entry['params']['L1'],
            'L2': entry['params']['L2'],
            'VDD': entry['params']['VDD'],
            'Temp': entry['params']['TEMP'],

        }

        features.update(model_params)
        log_path = entry['output']

        if os.path.exists(log_path):

            print(f"Processing run {entry['run']}... ({i+1}/{len(metadata)})", end='\r', flush=True)
            results = parse_spice_log(log_path)
```

```
            if results['simulation_success']:

                node_voltage = results.get('Node_Max_Voltage', {})
                node_nodecurrent = results.get('Node_Peak_NodeCurrent', {})
                flat_entry = {**features}
                voltage_nodes = ['vdd', 'gnd', 'in', 'out', 'target']
                current_nodes = ['vdd', 'gnd', 'in', 'out', 'target']

                for node in voltage_nodes:

                    if node == 'gnd':
                        flat_entry['V_gnd'] = 0.0

                    else:
                        flat_entry[f'V_{node}'] = node_voltage.get(node, float('nan'))

                for node in current_nodes:

                    if node == 'gnd':
                        flat_entry['I_gnd'] = node_nodecurrent.get('gnd', float('nan'))

                    else:
                        flat_entry[f'I_{node}'] = node_nodecurrent.get(node, float('nan'))

                dataset.append(flat_entry)

        else:
            print(f"Processing run {entry['run']}... X (Log file not found: {log_path})")

    print(f"Processing complete! Processed {len(metadata)} runs.                    ")

    if dataset:

        df = pd.DataFrame(dataset)

        for col in df.columns:

            if df[col].apply(lambda x: isinstance(x, dict)).any():
                df[col] = df[col].apply(lambda x: json.dumps(x) if isinstance(x, dict) else x)

        nunique = df.nunique(dropna=False)
        always_keep = {'Design'}

        if dataset:

            sample = dataset[0]

            for k in sample.keys():

                if k.startswith('V_') or k.startswith('I_'):
                    always_keep.add(k)

        constant_cols = [col for col in nunique[nunique <= 1].index.tolist() if col not in always_keep]

        if constant_cols:

            print(f"\nRemoving constant columns from dataset: {constant_cols}")
            df = df.drop(columns=constant_cols)

        for col in always_keep:

            if col not in df.columns:

                vals = [d.get(col, None) for d in dataset]
                df.insert(1, col, vals)

        dataset = df.to_dict(orient='records')

    return dataset, len(metadata), process_tag

def save_dataset(dataset, metadata_count, output_file='dataset.json'):

    with open(output_file, 'w') as f:
        json.dump(dataset, f, indent=2)

    if not dataset:
        print("\nNo successful simulation results found. Dataset is empty. Skipping CSV and summary statistics.")

        return

    df = pd.DataFrame(dataset)

    if 'Design' not in df.columns:

        design_val = None

        if len(dataset) > 0 and 'Design' in dataset[0]:
            design_val = dataset[0]['Design']

        if not design_val:
            design_val = os.environ.get('SPICE_DESIGN_NAME', '2inv')

        df.insert(1, 'Design', design_val)

    else:

        cols = list(df.columns)
        cols.remove('Design')
        cols.insert(1, 'Design')
        df = df[cols]

    csv_file = output_file.replace('.json', '.csv')
    df.to_csv(csv_file, index=False)
    successful_runs = len(dataset)
    print(f"\n--- Dataset Summary ---\n")
```

```
    new_sim_count = os.environ.get('NEW_SIM_COUNT')

    if new_sim_count is not None:
        print(f"New simulations run: {new_sim_count}")

    print(f"Total simulations: {metadata_count}")
    print(f"Successful: {successful_runs}")
    print(f"Failed: {metadata_count - successful_runs}")

    if successful_runs > 0:

        df = pd.DataFrame(dataset)

        if 'Target_Current' in df.columns:

            print(f"\nGNN Target: Target_Current (successful runs)")
            target_currents = df['Target_Current']
            print(f"Range: {target_currents.min():.2e} to {target_currents.max():.2e} A")
            print(f"Mean: {target_currents.mean():.2e} A")
            print(f"Std: {target_currents.std():.2e} A")

    print(f"\nDataset saved to: {output_file} and {csv_file}")

if __name__ == "__main__":

    RESULTS_DIR = "results"
    DATASET_DIR = "dataset"
    meta_path = os.path.join(RESULTS_DIR, 'metadata.json')

    if not os.path.exists(meta_path):
        print("Error: metadata.json not found. Run the simulations first!")

        exit(1)

    dataset, total_sims, process_tag = create_dataset()
    filename = f"{process_tag}_dataset.json" if process_tag else "dataset.json"
    save_dataset(dataset, total_sims, os.path.join(DATASET_DIR, filename))
```

predict.py

```
# ==========================================================================
# predict.py -- Run inference on new data using a saved model checkpoint
#
# Loads a trained GCN from results/model.pt and predicts branch currents
# for each row in a JSON dataset file. The checkpoint bundles the
# StandardScaler (fit during training) so features are scaled identically to
# what the model saw during training.
#
# Usage:
#   python predict.py --input ../dataset/dataset2.json
#   python predict.py --input ../dataset/my_new_data.json --output predictions.csv
#   python predict.py --input ../dataset/dataset2.json --checkpoint ../results/model.pt
# ==========================================================================

import argparse
import json
import os
import numpy as np
import pandas as pd
import torch
from types import SimpleNamespace
import models
from gan import GAN
from graph import Graph
from dataset import SKEW_CODES, OPTION_CODES

CURRENT_LABELS = ["I_vdd", "I_gnd", "I_in", "I_out", "I_target"]
I_TARGET_EDGE_INDEX = 3
CHECKPOINT_PATH = os.path.join(os.path.dirname(__file__), "..", "results", "model.pt")

def load_checkpoint(checkpoint_path, device):

    checkpoint = torch.load(checkpoint_path, map_location=device)
    config = SimpleNamespace(**checkpoint["config"])

    gan = GAN(


        checkpoint["embedding_dim"],
        config.hidden_dim,
        checkpoint["embedding_dim"],
        config.layers,
        heads=config.heads,

    )

    gan.load_state_dict(checkpoint["model_state_dict"])
    gan.to(device)
    gan.eval()

    return gan, config, checkpoint["label_log_mean"], checkpoint["label_log_std"], checkpoint["scaler"]

def encode_categoricals(row: dict) -> dict:

    row = dict(row)
    skew = row.pop("Skew", None)

    if skew is not None:
        row["SkewL"] = SKEW_CODES[skew[0]]
        row["SkewR"] = SKEW_CODES[skew[1]]

    if "Option" in row and isinstance(row["Option"], str):
```

```
        row["Option"] = OPTION_CODES[row["Option"]]

    return row

def scale_row(row: dict, scaler, feature_columns: list) -> dict:

    values = np.array([[row.get(c, 0.0) for c in feature_columns]], dtype=np.float32)
    scaled = scaler.transform(values)[0]

    return {c: float(scaled[i]) for i, c in enumerate(feature_columns)}

def predict_row(gan, config, label_log_mean, label_log_std, scaler, feature_columns, raw_row, device):

    row = encode_categoricals(raw_row)
    design = row.get("Design", "2inv")
    encoder = getattr(models, config.model + "_" + design)

    for field in CURRENT_LABELS:
        row[field] = 0.0

    if scaler is not None:
        scaled = scale_row(row, scaler, feature_columns)
        row.update(scaled)

    edges, X = encoder(row, design)
    graph = Graph(edges, X, config)
    A = graph.A.to(device)
    X_t = graph.X.to(device)

    with torch.no_grad():
        z = gan.encode(X_t, A)
        out = gan.decode(z, A).view(-1)

    i_target_pred = 10 ** (float(out[I_TARGET_EDGE_INDEX]) * label_log_std + label_log_mean)
    return i_target_pred

def get_feature_columns(scaler, data_sample: dict) -> list:

    drop = {"Design", "ID", "Skew"}  # dropped before scaler.fit in dataset.py
    cols = [k for k in data_sample.keys() if k not in drop]
    return cols

def main():

    parser = argparse.ArgumentParser(description="Run GCN current prediction inference")
    parser.add_argument("--input", required=True, help="Path to input JSON dataset")
    parser.add_argument("--output", default="predictions.csv", help="Output CSV file path")
    parser.add_argument("--checkpoint", default=CHECKPOINT_PATH, help="Path to model checkpoint (.pt)")
    args = parser.parse_args()
    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")
    print(f"Loading checkpoint from {args.checkpoint} ...")
    gcn, config, label_log_mean, label_log_std, scaler = load_checkpoint(args.checkpoint, device)
    print(f"  model={config.model}, layers={config.layers}, hidden={config.hidden_dim}, heads={config.heads}")
    print(f"  label_log_mean={label_log_mean:.4f}, label_log_std={label_log_std:.4f}")
    print(f"Loading data from {args.input} ...")

    with open(args.input) as f:
        data = json.load(f)

    print(f"  {len(data)} rows loaded")
    first_encoded = encode_categoricals(data[0])
    feature_columns = get_feature_columns(scaler, first_encoded)
    print("Running inference ...")
    rows_out = []

    for raw_row in data:

        i_target = predict_row(gcn, config, label_log_mean, label_log_std,
                               scaler, feature_columns, raw_row, device)

        rows_out.append({

            "ID": raw_row.get("ID"),
            "Design": raw_row.get("Design"),
            "I_target_pred": i_target,

        })

    df = pd.DataFrame(rows_out)
    df.to_csv(args.output, index=False)
    print(f"Predictions saved to {args.output}  ({len(df)} rows)")

if __name__ == "__main__":
    main()
```

run_sims.py

```
# ==========================================================================
# run_sims.py -- SPICE Simulation Automation
#
# Automates SPICE circuit simulations across process, voltage, and temperature (PVT)
# corners and device parameter sweeps. Generates simulation results, parses outputs,
# and creates datasets and matrices for machine learning (GNN) training. Handles
# cleaning, netlist selection, and robust path management for flexible execution.
#
# Usage:
#   python run_sims.py --design <design_name> [--dataset <name>] [--clean [all]]
#
# Example:
#   python run_sims.py --design 2inv --dataset dataset2 --clean
#   python run_sims.py --design 2inv --clean all
```

```
# ==============================================================================

import subprocess
import os
import random
import json
import time
import numpy as np
import argparse
import datetime
import re

script_dir = os.path.dirname(os.path.abspath(__file__))
project_root = os.path.abspath(os.path.join(script_dir, '..'))

if os.getcwd() != project_root:
    os.chdir(project_root)

seed = (int(time.time() * 1e6) ^ os.getpid() ^ int(datetime.datetime.now().microsecond)) % (2**32 - 1)
np.random.seed(seed)
random.seed(seed)

try:
    from python.parse_results import create_dataset, save_dataset, parse_spice_log

except ModuleNotFoundError:
    from parse_results import create_dataset, save_dataset, parse_spice_log

parser = argparse.ArgumentParser(description="Run SPICE simulations and generate dataset.")
parser.add_argument('--design', type=str, required=True, help='Design netlist name (without .sp)')
parser.add_argument('--dataset', type=str, default='dataset2', help='Dataset base name (writes <name>.json and <name>.csv)')

parser.add_argument(

    '--clean',
    nargs='?',
    const='design',
    choices=['design', 'all'],
    help='Clean results before running: use --clean for current design only, or --clean all for all results'

)

parser.add_argument('--debug', action='store_true', help='Do not trim log files; keep full NGSPICE output')
parser.add_argument('--task-id', type=int, default=None,
    help='SLURM array task ID for parallel execution (0-based index into model×pvt×skew combos)')
parser.add_argument('--count-tasks', action='store_true',
    help='Print total number of array tasks needed and exit (used by submit_sims.sh)')
parser.add_argument('--finalize', action='store_true',
    help='Merge per-task metadata and build dataset (run after array job completes)')
args = parser.parse_args()
dataset_name = os.path.splitext(args.dataset)[0]
os.system('cls' if os.name == 'nt' else 'clear')

NGSPICE = "ngspice"
BASE_SPICE = os.path.join("designs", "template.sp")
MODELS_DIR = "models"
CORNERS_DIR = "corners"
OUT_DIR = "results"
MATRICES_DIR = "matrices"
DATASET_DIR = "dataset"
SCRIPTS_DIR = "scripts"
NUM_SAMPLES = 7

os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(MATRICES_DIR, exist_ok=True)
os.makedirs(DATASET_DIR, exist_ok=True)

if args.task_id is None:

    if args.clean == 'all':
        print("Cleaning all results for fresh run...")

        for file in os.listdir(OUT_DIR):
            file_path = os.path.join(OUT_DIR, file)

            if os.path.isfile(file_path):
                os.remove(file_path)

    elif args.clean == 'design':

        print(f"Cleaning results for design '{args.design}'...")
        design_log_pattern = re.compile(rf"run_{re.escape(args.design)}_(\d+)\.log$")
        design_metadata = f"metadata_{args.design}.json"

        for file in os.listdir(OUT_DIR):
            file_path = os.path.join(OUT_DIR, file)

            if not os.path.isfile(file_path):
                continue

            if design_log_pattern.fullmatch(file) or file == design_metadata:
                os.remove(file_path)

DESIGN_NETLIST = os.path.join("designs", args.design + '.sp')

with open(BASE_SPICE) as f:

    for line in f:

        if line.strip().startswith('.include') and os.path.basename(DESIGN_NETLIST) in line:

            netlist_name = line.strip().split()[-1].replace('"', '')
            break

    else:
```

```
        netlist_name = os.path.basename(DESIGN_NETLIST)

print(f"Simulating {netlist_name} to create a dataset...\n")

PROCESS_PARAM_RANGES = {

    "22nm_HP.pm": {
        "WN1": (0.2e-6, 8e-6), "WP1": (0.2e-6, 12e-6),
        "WN2": (0.2e-6, 8e-6), "WP2": (0.2e-6, 12e-6),
        "L1": (20e-9, 200e-9), "L2": (20e-9, 200e-9),
        "VDD": (0.7, 1.3), "TEMP": (-20, 125)
    },

    "22nm_LP.pm": {
        "WN1": (0.2e-6, 8e-6), "WP1": (0.2e-6, 12e-6),
        "WN2": (0.2e-6, 8e-6), "WP2": (0.2e-6, 12e-6),
        "L1": (20e-9, 200e-9), "L2": (20e-9, 200e-9),
        "VDD": (0.7, 1.3), "TEMP": (-20, 125)
    },

    "32nm_HP.pm": {
        "WN1": (0.25e-6, 10e-6), "WP1": (0.25e-6, 15e-6),
        "WN2": (0.25e-6, 10e-6), "WP2": (0.25e-6, 15e-6),
        "L1": (30e-9, 250e-9), "L2": (30e-9, 250e-9),
        "VDD": (0.8, 1.4), "TEMP": (-20, 125)
    },

    "32nm_LP.pm": {
        "WN1": (0.25e-6, 10e-6), "WP1": (0.25e-6, 15e-6),
        "WN2": (0.25e-6, 10e-6), "WP2": (0.25e-6, 15e-6),
        "L1": (30e-9, 250e-9), "L2": (30e-9, 250e-9),
        "VDD": (0.8, 1.4), "TEMP": (-20, 125)
    },

    "45nm_HP.pm": {
        "WN1": (0.3e-6, 12e-6), "WP1": (0.3e-6, 18e-6),
        "WN2": (0.3e-6, 12e-6), "WP2": (0.3e-6, 18e-6),
        "L1": (40e-9, 300e-9), "L2": (40e-9, 300e-9),
        "VDD": (0.9, 1.5), "TEMP": (-20, 125)
    },

    "45nm_LP.pm": {
        "WN1": (0.3e-6, 12e-6), "WP1": (0.3e-6, 18e-6),
        "WN2": (0.3e-6, 12e-6), "WP2": (0.3e-6, 18e-6),
        "L1": (40e-9, 300e-9), "L2": (40e-9, 300e-9),
        "VDD": (0.9, 1.5), "TEMP": (-20, 125)
    },

    "65nm_bulk.pm": {
        "WN1": (0.4e-6, 15e-6), "WP1": (0.4e-6, 20e-6),
        "WN2": (0.4e-6, 15e-6), "WP2": (0.4e-6, 20e-6),
        "L1": (60e-9, 400e-9), "L2": (60e-9, 400e-9),
        "VDD": (1.0, 1.6), "TEMP": (-20, 125)
    },

    "90nm_bulk.pm": {
        "WN1": (0.5e-6, 20e-6), "WP1": (0.5e-6, 25e-6),
        "WN2": (0.5e-6, 20e-6), "WP2": (0.5e-6, 25e-6),
        "L1": (80e-9, 500e-9), "L2": (80e-9, 500e-9),
        "VDD": (1.1, 1.8), "TEMP": (-20, 125)
    },

    "130nm_bulk.pm": {
        "WN1": (0.6e-6, 25e-6), "WP1": (0.6e-6, 30e-6),
        "WN2": (0.6e-6, 25e-6), "WP2": (0.6e-6, 30e-6),
        "L1": (120e-9, 600e-9), "L2": (120e-9, 600e-9),
        "VDD": (1.2, 2.0), "TEMP": (-20, 125)
    }
}

def rand_param(lo, hi):
    return random.uniform(lo, hi)

def gen_params(model):

    model_name = os.path.basename(model)

    if model_name not in PROCESS_PARAM_RANGES:

        print(f"Warning: No parameter ranges defined for {model_name}, using 45nm defaults")
        model_name = "45nm_HP.pm"

    ranges = PROCESS_PARAM_RANGES[model_name]

    return {

        k: rand_param(v[0], v[1])
        for k, v in ranges.items()

    }

def write_netlist(params, model, pvt_corner, skew_corner, run_id, design_netlist):

    with open(BASE_SPICE) as f:
        txt = f.read()

    design_filename = os.path.basename(design_netlist)

    def replace_include(match):

        before = match.group(0)
        prefix = match.group(1)
        replaced = f'{prefix}{design_filename}"'
        return replaced
```

```
    txt = re.sub(r'(^\s*\.include\s+")([^"/\\]+\.sp)"', replace_include, txt, flags=re.MULTILINE)

    filled = txt.format(
        model=model,
        pvt_corner=pvt_corner,
        skew_corner=skew_corner,
        **params
    )

    filled = re.sub(r'^\s*\.print\s+tran\s+I\(VDD\)\s+I\(Vin\)\s*$', '', filled, flags=re.MULTILINE | re.IGNORECASE)
    filled = re.sub(r'^\s*\.print\s+tran\s+I\(vmeas_vdd\)[^\n]*$', '', filled, flags=re.MULTILINE | re.IGNORECASE)
    filled = re.sub(r'\n{2,}', '\n', filled)

    print_lines = [
        '.print tran V(vdd) V(in) V(target) V(out)',
        '.print tran I(vmeas_vdd) I(vmeas_in) I(vmeas_out) I(vmeas_target)',
        '.meas tran I_VDD_MAX MAX vdd#branch',
        '.meas tran I_Vin_MAX MAX vin#branch'
    ]

    if '.end' in filled:
        filled = filled.replace('.end', '\n' + '\n'.join(print_lines) + '\n.end')

    else:
        filled += '\n' + '\n'.join(print_lines) + '\n.end'

    fname = os.path.join("designs", f"run_{args.design}_{run_id}.sp")

    with open(fname, "w") as f:
        f.write(filled)

    return fname

def run_ngspice(netlist, out_file):

    cmd = [
        NGSPICE,
        "-b",
        "-o", out_file,
        netlist
    ]

    result = subprocess.run(cmd, check=True,
                            stdout=subprocess.DEVNULL,
                            stderr=subprocess.DEVNULL)

    if not args.debug:

        results = parse_spice_log(out_file)

        with open(out_file, 'w') as f:
            json.dump(results, f)

def regenerate_matrices():

    models = [
        os.path.join(MODELS_DIR, f)
        for f in sorted(os.listdir(MODELS_DIR))
        if f.endswith('.pm') and os.path.isfile(os.path.join(MODELS_DIR, f))
    ]

    pvt_corners = [
        os.path.join(CORNERS_DIR, f)
        for f in os.listdir(CORNERS_DIR)
        if f.endswith(".sp") and not f.startswith("skew_")
    ]

    skew_corners = [
        os.path.join(CORNERS_DIR, f)
        for f in os.listdir(CORNERS_DIR)
        if f.startswith("skew_") and f.endswith(".sp")
    ]

    if args.count_tasks:

        print(len(models) * len(pvt_corners) * len(skew_corners))
        exit(0)

    if args.task_id is not None:

        all_combos = [(m, p, s) for m in sorted(models) for p in sorted(pvt_corners) for s in sorted(skew_corners)]

        if args.task_id >= len(all_combos):

            print(f"Error: --task-id {args.task_id} out of range (0-{len(all_combos)-1})")

            exit(1)

        _m, _p, _s = all_combos[args.task_id]
        models       = [_m]
        pvt_corners  = [_p]
        skew_corners = [_s]

    if args.finalize:

        import glob as _glob
        task_files = sorted(_glob.glob(os.path.join(OUT_DIR, f'metadata_{args.design}_task_*.json')))
        print(f"Found {len(task_files)} per-task metadata files to merge...")
        all_entries = []

        for tf in task_files:

            with open(tf) as fh:

                try:
```

```
                d = json.load(fh)
                all_entries.extend(d if isinstance(d, list) else [d])

            except Exception:
                pass

    if all_entries:

        metadata_path = os.path.join(OUT_DIR, f'metadata_{args.design}.json')
        existing = []

        if os.path.exists(metadata_path):

            with open(metadata_path) as fh:

                try:
                    existing = json.load(fh)

                except Exception:
                    pass

        existing_tuples = set(tuple(sorted(e['params'].items())) for e in existing)
        new_entries = [e for e in all_entries if tuple(sorted(e['params'].items())) not in existing_tuples]
        combined = existing + new_entries

        with open(metadata_path, 'w') as fh:
            json.dump(combined, fh, indent=2)

        print(f"Merged {len(new_entries)} new entries from {len(task_files)} task files -> {metadata_path}")

        for tf in task_files:
            os.remove(tf)

    print("Parsing results and creating dataset...")
    dataset, total_sims = create_dataset()
    save_dataset(dataset, total_sims, os.path.join(DATASET_DIR, f'{dataset_name}.json'))
    print()
    exit(0)

if args.task_id is not None:
    run_id = args.task_id * NUM_SAMPLES

elif args.clean:
    run_id = 0

else:

    log_pattern = rf"run_{args.design}_(\\d+)\\.log"
    existing_logs = [f for f in os.listdir(OUT_DIR) if re.fullmatch(log_pattern, f)]

    if existing_logs:
        max_id = max(int(re.fullmatch(log_pattern, f).group(1)) for f in existing_logs)
        run_id = max_id + 1

    else:
        run_id = 0

total_simulations = len(models) * len(pvt_corners) * len(skew_corners) * NUM_SAMPLES
print(f"Starting {total_simulations} simulations...")

if total_simulations > 500:
    print(f"0/{total_simulations} simulations complete", end='', flush=True)

meta_data = []

existing_params_set = set()

if not args.clean:
    metadata_path = os.path.join(OUT_DIR, f"metadata_{args.design}.json")

    if os.path.exists(metadata_path):

        with open(metadata_path, "r") as f:

            try:

                prev_meta = json.load(f)
                for entry in prev_meta:

                    param_tuple = tuple(sorted(entry["params"].items()))
                    existing_params_set.add(param_tuple)

            except Exception:
                pass

start_time = time.time()

session_run_idx = 0

for model in models:

    for pvt_corner in pvt_corners:

        for skew_corner in skew_corners:

            model_name = os.path.basename(model)

            if model_name not in PROCESS_PARAM_RANGES:

                print(f"Warning: No parameter ranges defined for {model_name}, using 45nm defaults")
                model_name = "45nm_HP.pm"

            ranges = PROCESS_PARAM_RANGES[model_name]
            new_unique_params = set()
```

```
            max_attempts = NUM_SAMPLES * 100  # Avoid infinite loop
            attempts = 0

            while len(new_unique_params) < NUM_SAMPLES and attempts < max_attempts:

                params = {k: np.random.uniform(lo, hi) for k, (lo, hi) in ranges.items()}
                param_tuple = tuple(sorted(params.items()))
                attempts += 1

                if param_tuple in existing_params_set or param_tuple in new_unique_params:
                    continue

                new_unique_params.add(param_tuple)
                existing_params_set.add(param_tuple)

                netlist = write_netlist(
                    params, model, pvt_corner, skew_corner, run_id, DESIGN_NETLIST
                )

                out_file = os.path.join(
                    OUT_DIR,
                    f"run_{args.design}_{run_id}.log"
                )

                model_basename = os.path.basename(model).replace('.pm', '')
                pvt_name = os.path.basename(pvt_corner).replace('.sp', '')
                skew_name = os.path.basename(skew_corner).replace('skew_', '').replace('.sp', '')

                if '_' in model_basename:
                    process_node, process_option = model_basename.rsplit('_', 1)
                else:
                    process_node = model_basename
                    process_option = ""

                session_run_idx += 1
                if total_simulations > 500:
                    if (session_run_idx) % 100 == 0 or (session_run_idx) == total_simulations:
                        print(f"\r{session_run_idx}/{total_simulations} simulations complete", end='', flush=True)
                else:
                    print(
                        f"Run {session_run_idx:3d}/{total_simulations}: {process_node:8s} "
                        f"{process_option:4s} PVT:{pvt_name:8s} Skew:{skew_name:2s} | "
                        f"VDD={params['VDD']:.2f}V T={params['TEMP']:.0f}°C "
                        f"W1P={params.get('WP1', float('nan')):.2e} "
                        f"W1N={params.get('WN1', float('nan')):.2e} "
                        f"W2P={params.get('WP2', float('nan')):.2e} "
                        f"W2N={params.get('WN2', float('nan')):.2e}"
                    )

                run_ngspice(netlist, out_file)

                meta_data.append({
                    "run": run_id,
                    "design": args.design,
                    "model": model,
                    "pvt_corner": pvt_corner,
                    "skew_corner": skew_corner,
                    "params": params,
                    "output": out_file
                })

                try:
                    os.remove(netlist)
                except Exception as e:
                    print(f"Warning: Could not delete netlist {netlist}: {e}")

                run_id += 1

            if len(new_unique_params) < NUM_SAMPLES:
                print(
                    f"Warning: Only {len(new_unique_params)} unique parameter sets "
                    f"generated for {model_name}, {pvt_corner}, {skew_corner} "
                    f"after {attempts} attempts."
                )

end_time = time.time()
elapsed = end_time - start_time
avg_time = elapsed / total_simulations if total_simulations > 0 else 0

if total_simulations > 500:
    print()  # Print newline after progress overwrite

print(f"\nTotal simulation time: {elapsed:.2f} seconds")
print(f"Average time per simulation: {avg_time:.2f} seconds")

if args.task_id is not None:
    task_meta_path = os.path.join(OUT_DIR, f"metadata_{args.design}_task_{args.task_id}.json")
    with open(task_meta_path, "w") as f:
        json.dump(meta_data, f, indent=2)
else:
    metadata_path = os.path.join(OUT_DIR, f"metadata_{args.design}.json")
    if args.clean:
        with open(metadata_path, "w") as f:
            json.dump(meta_data, f, indent=2)
    else:
        prev_meta = []
        if os.path.exists(metadata_path):
            try:
                with open(metadata_path, "r") as f:
                    prev_meta = json.load(f)
            except Exception:
                prev_meta = []
        prev_param_tuples = set(tuple(sorted(entry["params"].items())) for entry in prev_meta)
        new_entries = [entry for entry in meta_data if tuple(sorted(entry["params"].items())) not in prev_param_tuples]
        combined_meta = prev_meta + new_entries
```

```
            with open(metadata_path, "w") as f:
                json.dump(combined_meta, f, indent=2)

    if total_simulations > 500:
        print()

    if args.task_id is None:
        num_existing = len(existing_params_set) if not args.clean else 0
        num_new = len(meta_data)
        import os as _os_env
        _os_env.environ['NEW_SIM_COUNT'] = str(num_new)
        _os_env.environ['TOTAL_SIM_COUNT'] = str(num_existing + num_new)
        print("Parsing results and creating dataset...")
        dataset, total_sims = create_dataset()
        save_dataset(dataset, total_sims, os.path.join(DATASET_DIR, f'{dataset_name}.json'))
        print()
        print()
```

sweep.py

```
# ==============================================================================
# sweep.py -- Hyperparameter Sweep Entry Point
#
# Defines the hyperparameter configuration and the main() training run.
# Data loading, the dataset class, loss functions, and the train/test loops
# all live in dataset.py. The GCN model architecture is in gcn.py.
#
# Runs training directly with a plain config object. W&B is used for logging
# via wandb.init but does not require sweep/agent API access.
# ==============================================================================

from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader
from types import SimpleNamespace
import os
import torch
import wandb
from dataset import circuit_dataset, data_frame, device, train, test, label_log_mean, label_log_std, scaler
from gan import GAN

print("[sweep] base imports done", flush=True)
print("[sweep] dataset imported", flush=True)
print("[sweep] GCN imported", flush=True)

config = SimpleNamespace(

    batch_size       = 32,
    model            = 'block',
    lr               = 0.0001,
    layers           = 3,
    hidden_dim       = 64,
    heads            = 4,
    test_size        = 0.3,
    epochs           = 100,
    edges_per_graph  = 7,
    target_edge_idx  = 3,
)

def main(config):

    run = wandb.init(

        entity="yosemitesamurai",
        project="CurrentPrediction",
        config=vars(config),

    )

    run.name = f"{config.hidden_dim}-width, {2 + config.layers}-layer, {config.heads}-heads"
    print(f"Starting run: {run.name}", flush=True)
    print(f"Device: {device}", flush=True)

    train_df, test_df = train_test_split(

        data_frame,
        test_size=config.test_size)

    train_dataset = circuit_dataset(train_df, config)
    test_dataset = circuit_dataset(test_df, config)
    print(f"[sweep] datasets created: {len(train_dataset)} train, {len(test_dataset)} test", flush=True)

    trainloader = DataLoader(

        train_dataset,
        batch_size=config.batch_size,
        shuffle=True)

    testloader = DataLoader(

        test_dataset,
        batch_size=config.batch_size,
        shuffle=False)

    embedding_dim = train_dataset[0][1].shape[1]
    gan = GAN(embedding_dim, config.hidden_dim, embedding_dim, config.layers, heads=config.heads)
    gcn.to(device)
    print(f"[sweep] model initialized, starting training...", flush=True)
    optimizer = torch.optim.Adam(params=gcn.parameters(), lr=config.lr)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', factor=0.5, patience=8, min_lr=1e-6)

    for epoch in range(config.epochs):
```

```
        gcn, optimizer, trainloss = train(gcn, optimizer, trainloader, config)
        testloss, testMRE, maxRE, minRE = test(gcn, testloader, config)
        scheduler.step(testloss)
        gan.load_state_dict(checkpoint["model_state_dict"])
        gan.to(device)
        gan.eval()
        current_lr = optimizer.param_groups[0]['lr']

        run.log({"Epoch": epoch + 1,
                 "Training Loss": trainloss,
                 "Avg Test Loss": testloss,
                 "Mean Relative Error (I_target)": testMRE,
                 "Max Relative Error (I_target)": maxRE,
                 "Min Relative Error (I_target)": minRE,
                 "LR": current_lr,
                 })

        print(f"Finished Epoch {epoch+1}", flush=True)
        print(f"Training Loss: {trainloss}", flush=True)
        print(f"Avg Test Loss {testloss}", flush=True)
        print(f"Mean Relative Error (I_target): {testMRE}", flush=True)
        print(f"LR: {current_lr}", flush=True)

    results_dir = os.path.join(os.path.dirname(__file__), "..", "results")
    os.makedirs(results_dir, exist_ok=True)

    checkpoint = {
        "model_state_dict": gcn.state_dict(),
        "config": vars(config),
        "label_log_mean": label_log_mean,
        "label_log_std": label_log_std,
        "embedding_dim": embedding_dim,
        "scaler": scaler,
    }

    model_path = os.path.join(results_dir, "model.pt")
    torch.save(checkpoint, model_path)
    print(f"Model saved to {model_path}", flush=True)
    run.finish()

if __name__ == '__main__':
    main(config)
```